

JAVASCRIPT PREMIUM NOTES

Detailed Topic Index / Table of Contents

Page numbers refer to the original note pages (1–120). The index is inserted before original page 1.

No.	Topic	Important Subtopics	Page
1	Basics of JavaScript	Console Object; Comments; Statements	1
2	Variables in JavaScript	Declaration, Assignment & Initialization; var, let & const; Scope; Naming Conventions	2
3	Data Types in JavaScript	Primitive & Reference Types; typeof Operator	5
4	Type Coercion	Implicit Type Coercion; Explicit Type Coercion	7
5	Dynamic & Forgiving Nature of JavaScript	Dynamically Typed Language; Forgiving Language	7
6	Operators in JavaScript	Operands and Operators; Expressions; Precedence; Associativity; Assignment; Arithmetic; Comparison; Logical; Increment/Decrement; Ternary; Bitwise	8
7	Truthy & Falsy Values	Falsy Values; Truthy Values; Examples	14
8	Short-Circuiting	&& Short Circuit; Short Circuit	15
9	Conditional Statements	if; if-else; if-else-if Ladder; Nested if-else; switch; Ternary	16
10	Loop Statements	for; while; do-while; nested loops; loop control	19
11	Functions	Function declaration; parameters/arguments; return; function scope; hoisting	21
12	Types of Functions	Function expression; arrow functions; anonymous functions; IIFE	24
13	Strings	String creation; properties; methods; template literals	25
14	Arrays	Array creation; indexing; multidimensional arrays; mutability; length	28
15	Array Methods	Adding/removing elements; searching; transforming; slice/splice and related methods	32
16	Sorting & Array Iteration	Sorting arrays; iteration methods; callback-based array operations	35
17	Objects	Object creation; properties; methods; accessing and updating object data	38

JAVASCRIPT PREMIUM NOTES

Detailed Topic Index / Table of Contents

Page numbers refer to the original note pages (1–120). The index is inserted before original page 1.

No.	Topic	Important Subtopics	Page
18	Object Types & Destructuring	Object types; destructuring assignment; nested destructuring	42
19	DOM Manipulation	DOM selection; changing content/styles; creating, adding and removing elements	45
20	Event Handling	Events; event listeners; event objects; common event handling techniques	61
21	Event Propagation & Delegation	Capturing; bubbling; propagation control; event delegation	65
22	Numbers	Number type; numeric operations; number conversion and handling	68
23	Math Object	Math properties and methods; common mathematical operations	70
24	Date Object	Creating dates; date/time methods; getting and setting date components	71
25	Programming Paradigms	Programming paradigms and major approaches to programming	78
26	Functional Programming	Functions as values; pure functions; higher-order functions; functional concepts	83
27	Object-Oriented Programming	Objects/classes; constructors; methods; OOP concepts	90
28	Four Pillars of OOP	Encapsulation; Abstraction; Inheritance; Polymorphism	95
29	APIs & JSON	APIs; XML; JSON; data exchange and parsing	99
30	Asynchronous Programming	Asynchronous execution; callbacks; CRUD-related async operations	104
31	Promises & Async/Await	Promises; states; chaining; async functions; await; error handling	108
32	Timers & Web Storage	setTimeout; setInterval; localStorage; sessionStorage	110
33	Spread & Rest Operators	Spread syntax; rest parameters; arrays/objects and function arguments	114
34	Modules in JavaScript	CommonJS; ES6 modules; named imports/exports; default imports/exports	117

JAVASCRIPT PREMIUM NOTES

Basics Of JavaScript

Console Object

The **console** is an object provided by the browser that helps developers:

- Print information
- Show errors
- Show warnings
- Display objects and tables
- Test values
- Debug JavaScript code

```
// Sample data
const user = {
  name: "Muhammad",
  age: 22,
  city: "Lahore"
};

// 1. console.log() - Displays normal information
console.log(user.name);

// 2. console.info() - Displays informational messages
console.info(user.name, user.age);

// 3. console.warn() - Displays a warning
console.warn("Warning! Something might be wrong.");

// 4. console.error() - Displays an error message
console.error("Error! Something went wrong.");

// 5. console.debug() - Displays debugging information
console.debug("Debugging the application...");
```

```
// 6. console.table() - Displays data in table format
console.table(user);

// 7. console.dir() - Displays an interactive object representation
console.dir(user);
```

Comments In JavaScript

Comments are used to add notes or explanations to your code. JavaScript ignores comments during execution. There are two types: single-line `//` and multi-line `/* ... */`.

```
// This is a single-line comment

/*
This is a
multi-line comment
*/
```

Statements In JavaScript

A statement is an instruction that tells JavaScript what to do. A statement terminator is usually a semicolon (;) used to mark the end of a statement.

```
let name = "Muhammad"; // Statement ends with ;

console.log(name); // Another statement

let age = 22; // Statement terminator ;
```

Variables In JavaScript

Variables

A variable is a named container used to store data in a program. The value stored inside a variable can be used, changed, or updated later.

Variable Declaration

Creating the variable

```
let age;
```

Variable Assignment

Giving a value to an existing variable

```
age = 22;
```

Variable Initialization

Giving a value when the variable is created

```
let age = 22;
```

Ways to Declare Variables

JavaScript provides three main keywords for declaring variables

- `var`
- `let`
- `const`

Feature	<code>var</code>	<code>let</code>	<code>const</code>
Can update value	Yes	Yes	No
Can redeclare	Yes	No	No
Scope	Function	Block	Block
Modern usage	Less recommended	Recommended	Recommended

```

// var: can be redeclared and reassigned
var name = "Muhammad";
var name = "Ali";
name = "Ahmed";

// let: can be reassigned but cannot be redeclared in the same scope
let age = 20;
age = 22;

// const: cannot be reassigned or redeclared
const country = "Pakistan";
// country = "India"; // Error

console.log(name);      // Ahmed
console.log(age);       // 22
console.log(country);   // Pakistan

```

Scope of Variables in JavaScript

Variable scope defines where a variable can be accessed or used in your code. The main types are

- Global Scope
- Function Scope
- Block Scope

```

// Global Scope - accessible everywhere
let globalVar = "I am global";

function example() {
    // Function Scope - accessible only inside this function
    let functionVar = "I am inside the function";
}

```

```

if (true) {
  // Block Scope - accessible only inside this block
  let blockVar = "I am inside the block";
  const blockConst = "I am also inside the block";

  console.log(globalVar);
  console.log(functionVar);
  console.log(blockVar);
  console.log(blockConst);
}

console.log(functionVar); // Accessible
// console.log(blockVar); // Error
}

example();

console.log(globalVar); // Accessible
// console.log(functionVar); // Error

```

Naming Conventions

Naming conventions are standard rules and styles used to name variables, functions, classes, and other identifiers. They make code clean, readable, and easier to understand.

- camelCase
- snake_case
- PascalCase

Data Types In JavaScript

Data types define the type of value that a variable can store. JavaScript has:

- Primitive data types
- Non-Primitive (Reference) data types

```

// Primitive Data Types
// NNSSBBU
let name = "Muhammad";           // String
let age = 22;                     // Number
let isStudent = true;             // Boolean
let data = undefined;             // Undefined
let emptyValue = null;            // Null
let uniqueId = Symbol("id");      // Symbol
let bigNumber = 123456789n;       // BigInt

// Non-Primitive / Reference Data Types
let user = { name: "Muhammad" }; // Object
let colors = ["Red", "Green"];    // Array
let greet = function () {};       // Function

console.log(typeof name);
console.log(typeof age);
console.log(typeof isStudent);
console.log(typeof user);
console.log(typeof colors);
console.log(typeof greet);

```

typeof Operator in JavaScript

The `typeof` operator is used to check the data type of a value or variable

```

let name = "Muhammad";
let age = 22;
let isStudent = true;
let value;
let empty = null;

console.log(typeof name); // "string"
console.log(typeof age);  // "number"
console.log(typeof isStudent); // "boolean"
console.log(typeof value); // "undefined"
console.log(typeof empty); // "object"

```

Type Coercion

Type coercion means the automatic or implicit conversion of a value from one data type to another by JavaScript when an operation is performed. There are two main types:

- Implicit Type Coercion
- Explicit Type Coercion

Implicit Type Coercion

Implicit coercion happens automatically. JavaScript converts the data type by itself during an operation.

```
console.log("5" + 2);    // "52" → Number converts to String
console.log("5" - 2);    // 3    → String converts to Number
console.log(true + 1);   // 2    → true converts to 1
```

Explicit Type Coercion

Explicit coercion happens when the developer manually converts a value using built-in functions such as `Number()`, `String()`, or `Boolean()`.

```
let value = "123";

console.log(Number(value)); // 123
console.log(String(123));  // "123"
console.log(Boolean(1));   // true
```

JavaScript Is a Dynamically Typed Language

JavaScript is dynamically typed, which means you do not need to specify the data type manually when declaring a variable. The type is automatically determined by JavaScript and can also change when you assign a new value.

```
let value = "Hello";

console.log(typeof value); // string

value = 100;

console.log(typeof value); // number

value = true;

console.log(typeof value); // boolean
```

JavaScript Is a Forgiving Language

JavaScript is called a forgiving language because it often tries to handle mistakes or automatically convert values instead of immediately showing an error. This is also known as type coercion

```
// JavaScript automatically converts "5" into a number
console.log("5" * 2); // 10

// JavaScript allows adding a number to a string
console.log("5" + 2); // "52"

// JavaScript converts false into 0
console.log(false + 5); // 5
```

Operators In JavaScript

Operands vs Operators

An **operator** is a symbol that performs an operation, while **operands** are the values or variables on which the operator performs that operation

```
let result = 10 + 5;
```

Here:

- `+` → Operator
- `10` and `5` → Operands
- `10 + 5` → Expression
- `result` → stores the final value 15

JavaScript Expressions

An expression is a combination of values, variables, and operators that produces a value.

```
let result = 10 + 5;
```

Operator Precedence

Precedence determines which operator is executed first when an expression contains multiple operators.

```
// () → Unary → * / % → + - → Comparison → Equality → && →  
|| → Assignment
```

```
let result = 10 + 5 * 2;
```

```
console.log(result); // 20
```

Operator Associativity

Associativity determines the direction in which operators with the same precedence are evaluated.

Left-to-Right Associativity

Most arithmetic operators work from left to right

```
let result = 20 - 5 - 3;
```

```
console.log(result); // 12
```

Right-to-Left Associativity

The assignment operator = works from right to left

```
let a;  
let b;  
  
a = b = 10;  
  
console.log(a); // 10  
console.log(b); // 10
```

Types Of Operators In JavaScript

The main types of JavaScript operators are

- Arithmetic Operator
- Assignment Operator
- Comparison Operator
- Logical Operator
- Bitwise Operator
- Increment & Decrement Operator
- Ternary Operator

Assignment Operator

Assignment operators are used to assign a value to a variable. The basic assignment operator is =, while compound assignment operators perform an operation and then assign the result back to the variable.

```
let x = 10;    // =   Assign  
  
x += 5;        // 15 → x = x + 5  
x -= 3;        // 12 → x = x - 3
```

```
x *= 2;      // 24 → x = x * 2
x /= 4;      // 6  → x = x / 4
x %= 4;      // 2  → x = x % 4
x **= 3;     // 8  → x = x ** 3
```

```
console.log(x); // 8
```

Arithmetic Operator

Arithmetic operators are used to perform mathematical calculations such as addition, subtraction, multiplication, division, and modulus.

```
let a = 10;
let b = 3;

console.log(a + b); // 13 → Addition
console.log(a - b); // 7  → Subtraction
console.log(a * b); // 30 → Multiplication
console.log(a / b); // 3.33 → Division
console.log(a % b); // 1   → Remainder (Modulus)
console.log(a ** b); // 1000 → Exponentiation
```

Comparison operators

Comparison operators are used to compare two values. They return a Boolean value: `true` or `false`.

```
let a = 10;
let b = 5;

console.log(a > b);    // true  → Greater than
console.log(a < b);    // false → Less than
console.log(a >= b);   // true  → Greater than or equal
console.log(a <= b);   // false → Less than or equal
console.log(a == b);   // false → Equal (loose)
console.log(a !== b);  // true  → Not equal (loose)
```

```
console.log(a === b); // false → Strictly equal
console.log(a !== b); // true → Strictly not equal
```

Logical operators

Logical operators are used to combine or reverse conditions. They return a Boolean value (`true` or `false`) and are commonly used with `if` statements.

```
let age = 22;
let hasID = true;

// && → AND: both conditions must be true
console.log(age >= 18 && hasID); // true

// || → OR: at least one condition must be true
console.log(age >= 18 || hasID); // true

// ! → NOT: reverses the result
console.log(!hasID); // false
```

Increment or Decrement Operator

Increment `++` increases a variable's value by 1, while Decrement `--` decreases it by 1. Both have prefix and postfix forms.

```
// ----- INCREMENT -----

let a = 5;

console.log(a++); // 5 → Post-increment: use first, then increase
console.log(a);   // 6

let b = 5;

console.log(++b); // 6 → Pre-increment: increase first, then use
```

```

console.log(b);    // 6

// ----- DECREMENT -----

let c = 5;

console.log(c--); // 5 → Post-decrement: use first, then decrease
console.log(c);   // 4

let d = 5;

console.log(--d); // 4 → Pre-decrement: decrease first, then use
console.log(d);   // 4

```

Ternary Operator

The ternary operator is a shorthand way of writing a simple `if...else` statement. It uses three parts - a condition, a value if the condition is true, and a value if the condition is false.

```

let age = 20;

let result = age >= 18 ? "Adult" : "Minor";

console.log(result); // Adult

```

Bitwise operators

Bitwise operators work on the binary representation (bits) of numbers. They convert numbers into binary, perform the operation bit by bit, and then return the decimal result.

```

let a = 5; // Binary: 0101
let b = 3; // Binary: 0011

```

```
console.log(a & b); // 1 → AND
console.log(a | b); // 7 → OR
console.log(a ^ b); // 6 → XOR
console.log(~a);    // -6 → NOT

console.log(a << 1); // 10 → Left Shift
console.log(a >> 1); // 2 → Right Shift
console.log(a >>> 1); // 2 → Zero-fill Right Shift
```

Truthy And Falsy Values

`truthy` and `falsey` values are values that JavaScript automatically treats as `true` or `false` when used in a Boolean context, such as an `if` condition.

Falsy Values

These values are treated as `false`

```
false
0
-0
0n
""
null
undefined
NaN
```

Truthy Values

Almost every other value is truthy, including

```
"Hello"
42
-10
[]
```

```
{}  
true
```

Example Code

```
let username = "";  
  
if (username) {  
    console.log("Username exists");  
} else {  
    console.log("Username is empty");  
}  
// Output: Username is empty
```

```
let name = "Muhammad";  
  
if (name) {  
    console.log("Name exists");  
} else {  
    console.log("Name is empty");  
}  
// Output: Name exists
```

Short Circuiting

Short-circuiting means JavaScript stops evaluating a logical expression as soon as the final result is already known. It mainly happens with `&&` and `||`.

```
// && stops when it finds a falsy value  
  
let result1 = "Hussain" && 21 && null && true && 123456789  
n;  
let result2 = "Hussain" && 21 && "Ilyas" && true && 1234567  
89n;  
console.log(result2);
```

```

let isAuthenticated = true;
let user = "Muhammad Hussain Ilyas;";
let result3 = isAuthenticated && user;
console.log(result3);

// || stops when it finds a truthy value

let result1 = "" || undefined || "Hussain" || null || 0n;
let result2 = "" || undefined || false || null || 0n;
console.log(result2);

```

Conditional Statements

Conditional statements are used to make decisions in a program based on whether a given condition is `true` or `false`. They control the flow of execution by allowing different blocks of code to run under different conditions.

The main conditional statements are:

- `if` Statement
- `if-else` Statement
- `if-else-if` Ladder
- Nested `if`
- `switch` statement
- Ternary operator

`if` Statement

The `if` statement executes a block of code only when the given condition is true.

```

int age = 20;

if (age >= 18) {
    cout << "You are eligible to vote.";
}

```

if-else Statement

The `if-else` statement executes one block when the condition is true and another block when it is false.

```
int age = 16;

if (age >= 18) {
    cout << "You are eligible to vote.";
} else {
    cout << "You are not eligible to vote.";
}
```

if-else-if Ladder

The `if-else-if` ladder is used when there are multiple conditions to check. The first condition that is true gets executed.

```
int marks = 75;

if (marks >= 90) {
    cout << "Grade A+";
} else if (marks >= 80) {
    cout << "Grade A";
} else if (marks >= 70) {
    cout << "Grade B";
} else {
    cout << "Grade C";
}
```

Nested if-else

A nested `if-else` means using an `if-else` statement inside another `if` or `else` block. It is used when we need to check a second condition depending on the result of the first condition.

```
int age = 20;
bool hasCard = true;
```

```

if (age >= 18) {
    if (hasCard) {
        cout << "Entry allowed.";
    } else {
        cout << "You need an ID card.";
    }
} else {
    cout << "You are not eligible for entry.";
}

```

switch Statement

The `switch` statement is a conditional control statement used to execute one block of code from multiple possible choices based on the value of an expression. It is often used as an alternative to a long `if-else-if` ladder when checking a single variable against different values. The `break` statement stops execution after a matching case, while `default` runs when no case matches.

```

int choice = 2;

switch (choice) {
    case 1:
        cout << "Add Record";
        break;

    case 2:
        cout << "View Record";
        break;

    case 3:
        cout << "Delete Record";
        break;

    default:
        cout << "Invalid Choice";
}

```

Ternary Operator

The ternary operator is a shorthand way of writing a simple `if...else` statement. It uses three parts - a condition, a value if the condition is true, and a value if the condition is false.

```
let age = 20;

let result = age >= 18 ? "Adult" : "Minor";

console.log(result); // Adult
```

Loop Statements

Loops are used to execute a block of code repeatedly as long as a specific condition is satisfied or for a specific number of times.

The main loops in JavaScript are

- `for` loop
- `while` loop
- `do...while` loop
- `for...of` loop
- `for...in` loop

`for` Loop

The `for` loop is used when you know how many times you want to execute a block of code.

```
for (let i = 1; i <= 5; i++) {
  console.log(i);
}
```

`while` Loop

The `while` loop repeatedly executes code as long as the condition is true.

```
let i = 1;

while (i <= 5) {
  console.log(i);
  i++;
}
```

do...while Loop

The `do...while` loop executes the code at least once, and then checks the condition.

```
let i = 1;

do {
  console.log(i);
  i++;
} while (i <= 5);
```

for...of Loop

The `for...of` loop is used to iterate over the values of iterable objects, such as arrays and strings.

```
let fruits = ["Apple", "Banana", "Mango"];

for (let fruit of fruits) {
  console.log(fruit);
}
```

for...in Loop

The `for...in` loop is mainly used to iterate over the keys/properties of an object.

```
let student = {
  name: "Ali",
```

```
    age: 20,  
    grade: "A"  
};  
  
for (let key in student) {  
    console.log(key + ": " + student[key]);  
}
```

Functions In JavaScript

A function in JavaScript is a reusable block of code designed to perform a specific task. Functions help make programs organized, reusable, and easier to maintain. A function can accept parameters, perform an operation, and optionally return a value.

Function Declaration

A function declaration defines a function using the `function` keyword.

```
// function Defination  
function greet() {  
    console.log("Hello, World!");  
}  
  
// Function Call  
greet();
```

Function with Parameters and Arguments

Parameters allow us to pass data to a function when calling it.

```
function greet(name, age) {  
    console.log("Name:", name);  
    console.log("Age:", age);  
}  
  
greet("Ali", 20);
```

Here, `name` and `age` are **parameters**, while `"Ali"` and `20` are **arguments**.

Function with Return Value

A function can use `return` to send a value back to the place where it was called.

```
function multiply(a, b) {  
    return a * b;  
}  
  
let result = multiply(5, 4);  
  
console.log(result);
```

Default Parameters

Default parameters allow us to assign a default value to a function parameter when no argument is provided or the argument is `undefined`.

```
function greet(name = "Guest") {  
    console.log("Hello " + name);  
}  
  
greet("Ali");  
greet();
```

Reference of a Function

A function reference means storing or passing a function without calling it. In JavaScript, functions are treated as values, so we can assign a function to a variable and call it using that variable.

```
function greet() {  
    console.log("Hello, World!");  
}  
  
let myFunction = greet; // Function reference
```

```
myFunction(); // Calling the function
```

Another Example

```
function greet() {  
    console.log("Hello!");  
}  
  
function execute(func) {  
    func();  
}  
  
execute(greet);
```

Function Hoisting

Function hoisting is a JavaScript behavior in which function declarations are moved to the top of their scope during the creation phase, allowing them to be called before they are written in the code. However, this works with function declarations, not function expressions or arrow functions in the same way.

```
greet(); // Function can be called before declaration  
  
function greet() {  
    console.log("Hello, World!");  
}
```

Arrow Function — No Hoisting

```
greet(); // ReferenceError  
  
const greet = () => {  
    console.log("Hello, World!");  
};
```

Types Of Functions

JavaScript provides different types of functions for performing specific tasks.

- Function Declaration
- Function Expression
- Arrow Function
- Anonymous Function
- IIFE (Immediately Invoked Function Expression)

Function Declaration

A function defined using the `function` keyword with a name.

```
function add(a, b) {  
    return a + b;  
}  
  
console.log(add(5, 3));
```

Function Expression

A function stored inside a variable.

```
const add = function(a, b) {  
    return a + b;  
};  
  
console.log(add(5, 3));
```

Arrow Function

A shorter way of writing functions using the `=>` syntax.

```
const add = (a, b) => {  
    return a + b;  
};
```

```
console.log(add(5, 3));
```

Anonymous Function

A function without a name, commonly used as a callback.

```
setTimeout(function() {  
    console.log("Hello!");  
}, 1000);
```

IIFE (Immediately Invoked Function Expression)

An IIFE is a function that is created and executed immediately.

```
(function() {  
    console.log("Function executed immediately!");  
})();
```

String In JavaScript

A String in JavaScript is a sequence of characters used to store and manipulate text. Strings can be created using single quotes `'...'`, double quotes `"..."`, or template literals `\`...\``; they can be initialized directly as literals or using the `String()` constructor. Strings use zero-based indexing `str[0]`, are immutable (their original value cannot be changed directly), and have built-in properties and methods for text manipulation.

Literal vs Constructor Method

```
let str1 = "Hello";           // String literal  
let str2 = new String("Hello"); // Constructor method
```

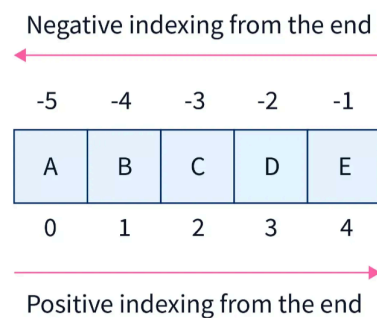
Single Quote & Double Quote

```
let name1 = 'Ali';  
let name2 = "Ahmed";
```

Indexing

Strings use zero-based indexing

```
let str = "Hello";  
  
str[0]; // H  
str[1]; // e  
str[4]; // o
```



Immutability

Strings cannot be changed directly after creation

```
let str = "Hello";  
str[0] = "Y";  
  
console.log(str); // Hello
```

Template Literals

Template literals use backticks and allow variables/expressions inside `${}`

```
let name = "Ali";  
let age = 20;
```

```
let msg = `My name is ${name} and I am ${age} years old.`;
```

String Length Property

It returns the number of characters (length) in a string

```
let str = "Hussain";  
console.log(str.length); // 7
```

String Methods

Important String Methods are following

```
let str = "Muhammad Hussain";  
  
// ===== 🚀 str.toUpperCase() =====  
// let res = str.toUpperCase();  
  
// ===== 🚀 str.toLowerCase() =====  
// let res = str.toLowerCase();  
  
// ===== 🚀 str.trim() =====  
// let res = str.trimStart();  
// let res = str.trimEnd();  
// let res = str.trim();  
  
// ===== 🚀 str.concat(str2, str3,...) =====  
// let str2 = "Ilyas";  
// let strSpace = " ";  
// let res = str.concat(strSpace, str2);  
  
// ===== 🚀 str.includes() =====  
// let res = str.includes("ain");  
  
// ===== 🚀 str.indexOf() =====  
// let res = str.indexOf("h");
```

```
// ===== 🚀 str.charAt() =====
// let res = str.charAt(2);

// ===== 🚀 str.replace(old, new) =====
// let res = str.replace("m", "M");

// ===== 🚀 str.replaceAll(old, new) =====
// let res = str.replaceAll("m", "M");

// ===== 🚀 str.slice(start, end) =====
// let res = str.slice(2);
// let res = str.slice(-7);
// let res = str.slice(2, 8);

// ===== 🚀 str.split() =====
// let res = str.split("");
// let res = str.split(" ");
// let res = str.split("h");
let res = str.split("ai");

console.log(res);
```

Arrays In JavaScript

An array is a **data structure** used to store multiple values in a single variable. The values are stored in an ordered way and can be accessed using their index.

```
let fruits = ["Apple", "Mango", "Banana"];

console.log(fruits);
// ["Apple", "Mango", "Banana"]
```

Literal Method

In the literal method, we create an array directly using square brackets `[]` and put values inside them.

```
let numbers = [10, 20, 30, 40];

console.log(numbers);
// [10, 20, 30, 40]
```

Constructor Method

In the constructor method, we use `new Array()` to create an array. The values are passed as arguments to the constructor.

```
let numbers = new Array(10, 20, 30);

console.log(numbers);
// [10, 20, 30]
```

`new Array(5).fill(0)` creates an array with a length of 5 containing all values 0, rather than an array containing the value 5.

```
let arr = new Array(5).fill(0);

console.log(arr.length);
// 5
```

Nesting with Arrays — 2-D Array

When an array contains another array inside it, it is called a nested array. A 2-D array is commonly used to represent data in rows and columns.

```
let matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
];
```

```
console.log(matrix);
```

We can access an element using two indexes

```
console.log(matrix[0][1]);  
// 2
```

Multi-Dimensional Array

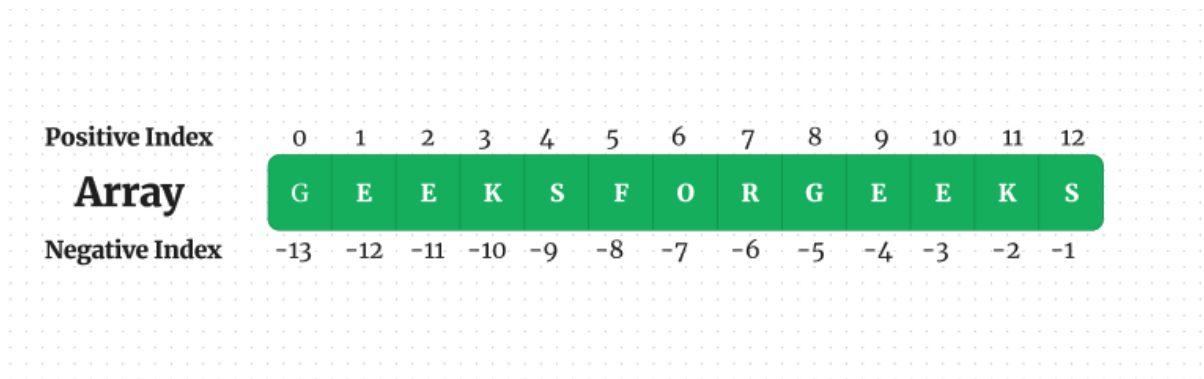
A multi-dimensional array is an array that contains arrays at multiple levels. For example, an array can contain 2-D arrays, creating a 3-D structure.

```
let data = [  
  [[1, 2], [3, 4]],  
  [[5, 6], [7, 8]]  
];  
  
console.log(data[1][0][1]);  
// 6
```

Indexing

very element in an array has an index number. JavaScript arrays use zero-based indexing, which means the first element has index `0`.

```
let fruits = ["Apple", "Mango", "Banana", "Orange"];  
  
console.log(fruits[0]);  
// Apple  
  
console.log(fruits[2]);  
// Banana
```



Mutability

Arrays are mutable, which means their elements can be changed after the array has been created.

```
let fruits = ["Apple", "Mango", "Banana"];

fruits[1] = "Orange";

console.log(fruits);
// ["Apple", "Orange", "Banana"]
```

We can also add or remove elements

```
let fruits = ["Apple", "Mango", "Banana"];

fruits.push("Grapes");

console.log(fruits);
// ["Apple", "Orange", "Banana", "Grapes"]
```

We can also delete elements

```
let fruits = ["Apple", "Mango", "Banana", "Orange"];

delete fruits[1];

console.log(fruits);
// ["Apple", empty, "Banana", "Orange"]
```

```
console.log(fruits.length);  
// 4
```

Why is `typeof Array` `"object"` ?

In JavaScript, an array is a special type of object. That's why `typeof` returns `"object"` when we check an array.

```
let fruits = ["Apple", "Mango", "Banana"];  
  
console.log(typeof fruits);  
// "object"
```

To specifically check whether a value is an array, we use `Array.isArray()`

```
let fruits = ["Apple", "Mango", "Banana"];  
  
console.log(Array.isArray(fruits));  
// true
```

Array `length` Property

The length property tells us the total number of elements in an array.

```
let fruits = ["Apple", "Mango", "Banana", "Orange"];  
  
console.log(fruits.length);  
// 4
```

Array Methods

Important String Methods are following

```
let arr = [2, 5, "MHI", 6, 5, 8];  
  
// ===== 💡 push(elem) =====  
// let a = arr.push("Ali");
```

```

// console.log(a); // return new length of array
// console.log(arr);

// ===== 💡 pop() =====
// let b = arr.pop();
// console.log(b); // return removed item
// console.log(arr);

// ===== 💡 unshift(elem) =====
// let c = arr.unshift("Hamid");
// console.log(c); // new length of array
// console.log(arr);

// ===== 💡 shift() =====
// let d = arr.shift();
// console.log(d); // return removed item
// console.log(arr);

// ===== 💡 slice(start, end) =====
// let newArr = arr.slice(1, 3);
// let newArr2 = arr.slice(-4, 3);
// console.log(newArr2);

// ===== 💡 splice(start, delete, elem1, elem2,...) =====
// let x = arr.splice(2, 0, "Hussain", 21);
// let x1 = arr.splice(2, 1, "Hussain", 21);
// let x2 = arr.splice(2, 2, "Hussain", 21);
// let x3 = arr.splice(2, 2);
// console.log(arr);
// console.log(x3);

// ===== 💡 concat(arr1, arr2,...) =====
// let arr1 = [1, 2, 3, 4];
// let arr2 = [5, 6, 7, 8];
// let newArr = arr.concat(arr1, arr2);
// console.log(newArr);

// ===== 💡 join(something) =====

```

```

// let str = arr.join("");
// let str1 = arr.join(" ");
// let str2 = arr.join("-");
// let str3 = arr.join(" AND ");
// console.log(str3);

// ===== 💡 toString() =====
// let str = arr.toString();
// console.log(str);

// ===== 💡 includes() =====
// let res = arr.includes("MHI");
// let res1 = arr.includes("mhi");
// console.log(res1);

// ===== 💡 indexOf(elem) =====
// let index = arr.indexOf("MHI");
// console.log(index);

// ===== 💡 lastIndexOf(elem) =====
// let index = arr.lastIndexOf(5);
// console.log(index);

// ===== 💡 reverse() =====
// let res = arr.reverse();
// console.log(arr);
// console.log(res);

// ===== 💡 find(func) =====
// let result = arr.find((item) => {
//   return item === "MHI";
// });
// console.log(result);

// ===== 💡 flat(level) =====
// let marks = [1, 2, [3, [4, 5, 6], 7, 8], 9];
// console.log(marks);
// let newArr = marks.flat();

```

```
// let newArr1 = marks.flat(1);
// let newArr2 = marks.flat(2);
// let newArr3 = marks.flat(Infinity);
// console.log(newArr3);
```

Sorting An Array

It works on **Tim Sort** under the hood (a combination of **merge sort** and **insertion sort**)

By default, `sort()` converts elements to strings and sorts them **alphabetically** (Unicode order) or **Lexicographical order**.

Return value of a function	Meaning	Effect on order
Negative <code>< 0</code>	<code>a</code> should come before <code>b</code>	keep order as-it-is
Zero <code>=== 0</code>	<code>a</code> and <code>b</code> are equal	keep their order (stable)
Positive <code>> 0</code>	<code>a</code> should come after <code>b</code>	swap <code>a</code> and <code>b</code>

String Array

```
let fruits = ["Banana", "Apple", "Orange", "Mango"];

fruits.sort();

console.log(fruits);
// ["Apple", "Banana", "Mango", "Orange"]
```

Number Array

```
// Ascending Order

let numbers = [10, 5, 100, 25, 2];

numbers.sort((a, b) => a - b);

console.log(numbers);
```

```
// Descending Order

let numbers = [10, 5, 100, 25, 2];

numbers.sort((a, b) => b - a);

console.log(numbers);
```

Array Iteration Methods

Array iteration methods are built-in JavaScript methods used to go through each element of an array and perform an operation on it. They make it easier to process, search, transform, or filter array elements without using traditional `for` loops.

- `forEach()` method
- `map()` method
- `filter()` method
- `reduce()` method

`forEach()` Method

`forEach()` executes a function once for every element in the array. It is mainly used when you want to perform an action on each element.

```
let numbers = [10, 20, 30, 40];

numbers.forEach(function (num) {
  console.log(num);
});

// 10
// 20
// 30
// 40
```

map() Method

`map()` creates a new array by applying a function to every element of the original array.

```
let numbers = [1, 2, 3, 4];

let squares = numbers.map((num, index, arr) => {
  return num * num;
});

console.log(squares);
// [1, 4, 9, 16]
```

filter() Method

`filter()` creates a new array containing only the elements that satisfy a condition.

```
let numbers = [10, 15, 20, 25, 30];

let evenNumbers = numbers.filter((num, index, arr) => {
  return num % 2 === 0;
});

console.log(evenNumbers);
// [10, 20, 30]
```

reduce() Method

`reduce()` processes all elements and reduces them to one final value, such as a sum, product, or total.

```
let numbers = [10, 20, 30, 40];

let sum = numbers.reduce((acc, num, index, arr) => {
  return acc + num;
}, 0);
```

```
console.log(sum);  
// 100
```

Objects In JavaScript

An object in JavaScript is a collection of **key-value pairs** used to store related data and functionality together. Keys are usually called **properties**, while functions stored inside an object are called **methods**. Objects are **reference-based** and are **mutable**, meaning their **properties** can be changed after creation.

Object Literal Method

The object literal is the simplest and most common way to create an object. You directly write the object using `{ }` and provide its properties and values.

```
const person = {  
  name: "Ahmed",  
  age: 22,  
  city: "Lahore",  
  greet: function() {  
    console.log("Hello!");  
  }  
};  
  
console.log(person.name); // Ahmed  
person.greet(); // Hello!
```

Constructor Method

A **constructor method** is useful when you want to create multiple objects with the same structure. The traditional way is to use a constructor function with `new`

```
let student = new Object({  
  name: "hussain",  
  age: 21,
```

```
greet: function () {  
  console.log("Hye! Hussain");  
},  
});
```

Mutability

Objects in JavaScript are **mutable**, which means their properties can be changed after the object is created.

```
const student = {  
  name: "Hussain",  
  age: 20  
};  
  
// Change property  
student.age = 21;  
  
// Add property  
student.city = "Lahore";  
  
console.log(student);  
// { name: "Hussain", age: 21, city: "Lahore" }  
  
// Delete property  
delete student.city;  
  
console.log(student);  
// { name: "Hussain", age: 21 }
```

this Keyword

The `this` keyword refers to the object that is calling the method in a normal object method.

```
const student = {  
  name: "Ali",  
  age: 20,
```

```
    introduce() {
        console.log("My name is " + this.name);
        console.log("My age is " + this.age);
    }
};

student.introduce();
```

Accessing Object Properties

JavaScript provides two main ways to access object properties: Dot Notation `.` and Bracket Notation `[]`.

Dot Notation

```
const student = {
    name: "Ali",
    age: 20,
    city: "Lahore"
};

console.log(student.name); // Ali
console.log(student.age);  // 20
console.log(student.city); // Lahore
```

Bracket Notation

```
const student = {
    name: "Ali",
    age: 20,
    city: "Lahore"
};

console.log(student["name"]); // Ali
```

```
console.log(student["age"]); // 20
console.log(student["city"]); // Lahore
```

Objects Methods

```
// let obj = {
//   name: "alpha",
//   age: 21,
// };

// ===== 💡 Object.keys(obj) =====
// let keysArr = Object.keys(obj);

// ===== 💡 Object.values(obj) =====
// let valuesArr = Object.values(obj);

// ===== 💡 Object.entries(obj) =====
// let entriesArr = Object.entries(obj);

// ===== 💡 Object.assign(target, source) =====
// const obj2 = {};
// let newObj = Object.assign(obj2, obj);
// console.log(newObj);
// console.log(obj2);

// ===== 💡 Object.freeze(obj) =====
// Object.freeze(obj);
// obj.city = "lahore"; // ❌ adding new property
// delete obj.age; // ❌ delete existing property
// obj.age = 22; // ❌ update existing property

// ===== 💡 Object.seal(obj) =====
// Object.seal(obj);
// obj.city = "lahore"; // ❌ adding new property
// delete obj.age; // ❌ delete existing property
// obj.age = 22; // ✔ update existing property
```

```
// ===== 💡 objName.hasOwnProperty(key) =====  
// console.log(obj.hasOwnProperty("age"));  
// console.log(obj.hasOwnProperty("city"));
```

Singleton Object

A singleton object is a single object created directly, usually using an object literal. It is used when you only need one specific object.

```
const student = {  
  name: "Ali",  
  age: 20,  
  course: "JavaScript"  
};  
  
console.log(student.name); // Ali
```

Class-Based Object

A class-based object is created from a class, which acts as a blueprint for creating multiple objects with the same structure and behavior.

```
class Student {  
  constructor(name, age, course) {  
    this.name = name;  
    this.age = age;  
    this.course = course;  
  }  
  
  study() {  
    console.log(this.name + " is studying.");  
  }  
}  
  
const student1 = new Student("Hussain", 20, "JavaScript");  
const student2 = new Student("Suleman", 22, "Python");  
  
console.log(student1.name); // Hussain
```

```
console.log(student2.name); // Suleman

student1.study(); // Hussain is studying.
```

Array Of Objects

An array whose values are objects is called an Array of Objects.

```
const students = [
  {
    name: "Ali",
    age: 20,
    course: "JavaScript"
  },
  {
    name: "Ahmed",
    age: 22,
    course: "Python"
  },
  {
    name: "Usman",
    age: 21,
    course: "C++"
  }
];
```

Destructuring in JavaScript

Destructuring is a JavaScript feature that allows you to extract values from objects or arrays and store them directly into variables. It makes code shorter, cleaner, and easier to read.

Object Destructuring

Instead of accessing each property separately,

```
const student = {
  name: "Ali",
```

```
    age: 20,  
    city: "Lahore"  
};  
  
const name = student.name;  
const age = student.age;  
  
console.log(name); // Ali  
console.log(age);  // 20
```

We can use object destructuring.

```
const student = {  
  name: "Ali",  
  age: 20,  
  city: "Lahore"  
};  
  
const { name, age } = student;  
  
console.log(name); // Ali  
console.log(age);  // 20
```

You can provide a **default value** if the property doesn't exist

```
const student = {  
  name: "Ali"  
};  
  
const { name, age = 20 } = student;  
  
console.log(name); // Ali  
console.log(age);  // 20
```

You can give a property a different variable name (**Renaming Variable**) using

```
: .
```

```
const student = {  
  name: "Ali",  
  age: 20  
};  
  
const { name: studentName, age: studentAge } = student;  
  
console.log(studentName); // Ali  
console.log(studentAge);  // 20
```

Array Destructuring

Destructuring also works with arrays. Here, values are extracted according to their position/index.

```
const colors = ["Red", "Green", "Blue"];  
  
const [first, second, third] = colors;  
  
console.log(first); // Red  
console.log(second); // Green  
console.log(third); // Blue
```

You can **skip values** using an empty comma

```
const numbers = [10, 20, 30];  
  
const [first, , third] = numbers;  
  
console.log(first); // 10  
console.log(third); // 30
```

DOM Manipulation



DOM stands for Document Object Model

When a browser loads an HTML document, it converts the HTML into a tree-like structure. This structure is called the DOM.

```
<!DOCTYPE html>
<html>
  <head>
    <title>My Website</title>
  </head>

  <body>
    <h1 id="heading">Hello World</h1>
  </body>
</html>
```

The browser represents it approximately like this

```
Document
  |
  HTML
  /  \
HEAD  BODY
  |    |
TITLE H1
  |    |
Text  Text
```

Node

A Node is any individual object in the DOM (Document Object Model). When HTML is converted into the DOM, everything becomes a node.

```
<div>Hello</div>
```

Here

- `<div>` → Element Node
- `"Hello"` → Text Node

Types of Nodes

The most important DOM node types are

Node Type	Meaning	Example
Element Node	HTML elements	<code><div></code> , <code><p></code> , <code><h1></code>
Text Node	Text inside elements	<code>Hello</code>
Attribute Node	HTML attributes	<code>id="box"</code>
Comment Node	HTML comments	<code><!-- comment --></code>
Document Node	Entire HTML document	<code>document</code>

NodeList

A `NodeList` is a collection/list of DOM nodes returned by some DOM methods.

```
let nodes = document.querySelectorAll("p");
```

If there are 3 `<p>` elements, `nodes` contains those 3 nodes. You can access them using

```
nodes[0]  
nodes[1]  
nodes[2]
```

HTMLCollection

An `HTMLCollection` is a collection of HTML elements returned by certain DOM methods.

```
let elements = document.getElementsByTagName("p");
```

If HTML has 3 `<p>` elements, `elements` will contain those 3 elements. You can access them using

```
elements[0]  
elements[1]  
elements[2]
```

HTMLCollection VS NodeList

HTMLCollection	NodeList
Contains HTML elements only	Can contain different types of nodes
Example: <code><p></code> , <code><div></code>	Element, Text, Comment, etc.
Commonly returned by <code>getElementsByName()</code>	Commonly returned by <code>querySelectorAll()</code>
Usually live	<code>querySelectorAll()</code> returns a static NodeList

Five(5) Phases Of DOM Manipulation

- DOM
- Selecting HTML elements
- How to change / manipulate HTML
- How to change / manipulate CSS
- Event Listeners (Event handling)

The document object

Almost everything related to DOM manipulation starts with

```
console.log(document);  
console.log(document.body);  
console.log(document.head);  
console.log(document.title);
```

You can even change the title

```
document.title = "My Website";
```

Selecting Elements

`getElementById()`

Selects an element using its `id`.

```
<h1 id="heading">Hello World</h1>

<script>
const heading = document.getElementById("heading");

console.log(heading);
</script>
```

`querySelector()`

Selects the first matching element.

```
<h1 id="main-heading">Heading</h1>
<p class="text">Paragraph</p>

<script>
const heading = document.querySelector("#text");
const paragraph = document.querySelector(".text");

console.log(heading);
console.log(paragraph);
</script>
```

`querySelectorAll()`

Selects all matching elements.

```
<p class="text">First</p>
<p class="text">Second</p>
<p class="text">Third</p>

<script>
const paragraphs = document.querySelectorAll(".text");
```

```
console.log(paragraphs);  
</script>
```

getElementsByClassName()

used to select all HTML elements that have a specific class name.

```
<p class="text">Second</p>  
<p class="text">Third</p>  
  
<script>  
const paragraphs = document.getElementsByClassName("text");  
  
console.log(paragraphs);  
</script>
```

getElementsByTagName()

selects all HTML elements having a specific tag name.

```
<p>Second</p>  
<p>Third</p>  
  
<script>  
const paragraphs = document.getElementsByTagName("p");  
  
console.log(paragraphs);  
</script>
```

DOM Traversing (Walking)

DOM Traversing means moving from one HTML element to another element through the DOM tree.

Walking Through Nodes

```

<!DOCTYPE html>
<html>
<body>
  <div id="parent">
    <h1 id="heading">Hello</h1>
    <p id="para">Welcome</p>
  </div>

  <script>
    let parent = document.querySelector("#parent");
    let heading = document.querySelector("#heading");

    // Parent node
    console.log("Parent Node:");
    console.log(heading.parentNode);

    // All child nodes
    console.log("Child Nodes:");
    console.log(parent.childNodes);

    // First child node
    console.log("First Child:");
    console.log(parent.firstChild);

    // Last child node
    console.log("Last Child:");
    console.log(parent.lastChild);

    // Next sibling node
    console.log("Next Sibling:");
    console.log(heading.nextSibling);

    // Previous sibling node
    console.log("Previous Sibling:");
    console.log(heading.previousSibling);
  </script>

```

```
</body>
</html>
```

Walking Through Elements

```
<!DOCTYPE html>
<html>
<head>
  <title>DOM Traversing</title>
</head>

<body>
  <div id="parent">
    <h1 id="heading">DOM Traversing</h1>
    <p id="first">First Paragraph</p>
    <p id="second">Second Paragraph</p>
    <button id="btn">Click Me</button>
  </div>

  <script>
    // Select Parent
    let parent = document.querySelector("#parent");

    // Select First Paragraph
    let firstPara = document.querySelector("#first");

    // =====
    // 1. Parent Traversing
    // =====

    console.log("Parent:");
    console.log(firstPara.parentElement);

    // =====
    // 2. Children Traversing
    // =====
```

```

    console.log("All Children:");
    console.log(parent.children);

    // =====
    // 3. First Child
    // =====

    console.log("First Child:");
    console.log(parent.firstChild);

    // =====
    // 4. Last Child
    // =====

    console.log("Last Child:");
    console.log(parent.lastElementChild);

    // =====
    // 5. Next Sibling
    // =====

    console.log("Next Sibling:");
    console.log(firstPara.nextElementSibling);

    // =====
    // 6. Previous Sibling
    // =====

    console.log("Previous Sibling:");
    console.log(firstPara.previousElementSibling);
</script>
</body>
</html>

```

Finding the Closest Ancestor — `closest()`

The `closest()` method is used to find the nearest ancestor element that matches a given CSS selector.

```
<div class="card">
  <section class="card">
    <button id="btn">Buy Now</button>
  </section>
</div>

<script>
  let button = document.querySelector("#btn");
  let card = button.closest(".card");
  console.log(card);
</script>
```

Manipulating HTML

Properties

tagName

The `tagName` property returns the name of an HTML element.

```
<h1 id="heading">Hello World</h1>

<script>
  let heading = document.querySelector("#heading");

  console.log(heading.tagName);
</script>
```

nodeName

The `nodeName` property returns the name of a DOM node.

```
<h1 id="heading">Hello World</h1>

<script>
  let heading = document.querySelector("#heading");
```

```
console.log(heading.nodeName);  
</script>
```

Changing HTML Content

There are three properties you should understand carefully

- `textContent`
- `innerText`
- `innerHTML`

textContent

```
<h1 id="title">Hello</h1>  
  
<script>  
  const title = document.querySelector("#title");  
  
  title.textContent = "Hello World";  
</script>
```

innerHTML

```
<h1 id="title">Hello</h1>  
  
<script>  
  const title = document.querySelector("#title");  
  
  title.innerHTML = "<b>Hello World</b>";  
</script>
```

innerText

```
<h1 id="title">Hello</h1>  
  
<script>  
  const title = document.querySelector("#title");
```

```
title.innerText = "Hello World";  
</script>
```

Manipulating Attributes

Manipulating attributes means getting, setting, changing, or removing HTML element attributes using JavaScript.

```
<a id="link" href="https://google.com">Google</a>  
  
<script>  
  let link = document.querySelector("#link");  
  
  // Get  
  console.log(link.getAttribute("href"));  
  
  // Change  
  link.setAttribute("href", "https://youtube.com");  
  
  // Check  
  console.log(link.hasAttribute("href"));  
  
  // Remove  
  link.removeAttribute("href");  
</script>
```

Creating Elements

`createElement()`

The `document.createElement()` method is used to create a new HTML element.

```
let heading = document.createElement("h1");  
console.log(heading);
```

Inserting Elements

These are the following methods are used to insert elements.

- `append()`
- `prepend()`
- `after()`
- `before()`
- `insertAdjacentElement()`
- `insertBefore()`
- `appendChild()`

Suppose we have

```
<div id="parent">
  <p id="child">Hello</p>
</div>
```

And JavaScript

```
let parent = document.querySelector("#parent");
let child = document.querySelector("#child");

let newElem = document.createElement("span");
newElem.textContent = "New Element";
```

`append()`

Adds an element **at the end inside** the parent.

```
parent.append(newElem);
```

`prepend()`

Adds an element at the beginning inside the parent.

```
parent.prepend(newElem);
```

`after()`

Adds an element outside and immediately after the selected element.

```
child.after(newElem);
```

before()

Adds an element outside and immediately before the selected element.

```
child.before(newElem);
```

insertAdjacentElement("position", element)

This method allows you to specify exactly where to insert the element.

It has 4 positions

- `beforebegin`
- `afterbegin`
- `beforeend`
- `afterend`

```
child.insertAdjacentElement("beforebegin", newElem);
```

insertBefore(newNode, referenceNode)

Inserts a new node before a specific child node.

```
parent.insertBefore(newElem, child);
```

appendChild()

Adds a node at the end inside the parent.

```
parent.appendChild(newElem);
```

Replacing Elements

Main methods

- `replaceWith()`
- `replaceChild()`

replaceWith()

Replaces the selected element with a new element.

```
let oldElem = document.querySelector("#old");

let newElem = document.createElement("p");
newElem.textContent = "New Element";

oldElem.replaceWith(newElem);
```

replaceChild()

Replaces a specific child element with a new element.

```
let parent = document.querySelector("#parent");
let oldElem = document.querySelector("#old");

let newElem = document.createElement("p");
newElem.textContent = "New Element";

parent.replaceChild(newElem, oldElem);
```

Deleting Elements

Main methods

- `remove()`
- `removeChild()`

remove()

Removes the selected element directly.

```
let elem = document.querySelector("#child");

elem.remove();
```

removeChild()

Removes a specific child from its parent.

```
let parent = document.querySelector("#parent");
let child = document.querySelector("#child");

parent.removeChild(child);
```

Manipulating CSS

CSS Manipulation in DOM means changing the CSS styles of HTML elements using JavaScript.

For example, suppose we have

```
<p id="para">Hello World</p>
```

First, we select the element

```
let p = document.querySelector("#para");
```

Using `setAttribute()` Method

The `setAttribute()` method can be used to add or change an HTML attribute.

```
p.setAttribute("class", "change-bg");

p.setAttribute("style", "padding: 10px; color: blue;");
```

Using `style` Property

This is one of the easiest and most common ways to change CSS with JavaScript.

```
p.style.backgroundColor = "blue";
p.style.color = "white";
```

Using `cssText` Property

`cssText` allows you to set multiple CSS properties at once.

```
p.style.cssText = "font-size: 30px; color: red; background-color: black;";
```

Using `className` Property

Instead of directly changing CSS, we can add a CSS class to an element and let the CSS class define the styling.

Suppose we have

```
.add-padding {  
  padding: 10px;  
}  
  
.change-color {  
  color: red;  
}  
  
.change-bg {  
  background-color: black;  
}
```

Then JavaScript

```
p.className = "add-padding change-color change-bg";
```

Using `classList` Property

`classList` allows us to manage individual classes without replacing all existing classes.

```
let classes = p.classList;
```

It contains four methods as following

- `add()`
- `remove()`
- `toggle()`

- `contains()`

```
let p = document.querySelector("p");

p.classList.add("change-bg");           // Add
p.classList.remove("change-bg");        // Remove
p.classList.toggle("change-bg");        // Add/Remove
console.log(p.classList.contains("change-bg")); // Check
```

Event Handling

Event handling means detecting and responding to actions that happen in a web page, such as a click, key press, form submission, mouse movement, or touch.

Inline Method (Inline JavaScript)

The event handler is written directly inside the HTML element using an event attribute.

```
<button onclick="alert('Button clicked!')">Click Me</button>
```

Property Method

The event is assigned to an element's event property.

```
let btn = document.querySelector("button");

btn.onclick = function () {
    alert("Button clicked!");
};
```

Listener Method

The recommended and most flexible method is `addEventListener()`.

```
let btn = document.querySelector("button");

btn.addEventListener("click", function () {
    alert("Button clicked!");
});
```

Remove Event Listener

`removeEventListener()` removes an event listener that was previously added.



The function must be stored in a variable/reference; you cannot normally remove an anonymous function by writing another identical function.

```
function handleClick() {
    console.log("Clicked!");
}

btn.addEventListener("click", handleClick);

// Remove the listener
btn.removeEventListener("click", handleClick);
```

Event Object

The event object contains information about the event that occurred.

It is commonly represented by `event` or `e`.

```
btn.addEventListener("click", function (event) {
    console.log(event);
    console.log(event.type);
    console.log(event.target);
});
```

Common properties/methods

- `event.type` → type of event
- `event.target` → element that triggered the event
- `event.currentTarget` → element whose listener is running
- `event.preventDefault()` → prevents default browser behavior
- `event.stopPropagation()` → stops event propagation

Mouse Events

Common mouse events include

Event	Purpose
<code>click</code>	Element is clicked
<code>dblclick</code>	Element is double-clicked
<code>mousedown</code>	Mouse button is pressed
<code>mouseup</code>	Mouse button is released
<code>mousemove</code>	Mouse moves
<code>mouseenter</code>	Mouse enters an element
<code>mouseleave</code>	Mouse leaves an element
<code>mouseover</code>	Mouse moves over an element
<code>mouseout</code>	Mouse moves out of an element
<code>contextmenu</code>	Right-click occurs

Keyboard Events

Keyboard events occur when the user interacts with the keyboard.

Event	Purpose
<code>keydown</code>	Key is pressed
<code>keyup</code>	Key is released
<code>keypress</code>	Older event for key presses - generally avoid it

Form Events

Common form-related events

Event	Purpose
-------	---------

submit	Form is submitted
reset	Form is reset
input	Input value changes
change	Value changes and is committed
focus	Element receives focus
blur	Element loses focus
select	Text is selected

Window Events

Window events are related to the browser window/page.

Event	Purpose
load	Page/resources finish loading
DOMContentLoaded	HTML document is loaded and parsed
resize	Browser window is resized
scroll	Page is scrolled

Clipboard Events

Event	Purpose
copy	Content is copied
cut	Content is cut
paste	Content is paste



Do You Know About The **\$0** Magic in Google Chrome Inspect Tool

Event Lifecycle

Event Propagation

When an event happens on a nested element, it travels through the DOM in 3 phases

Capturing

Event travels from parent → child.

```
parent.addEventListener("click", handler, true);
```

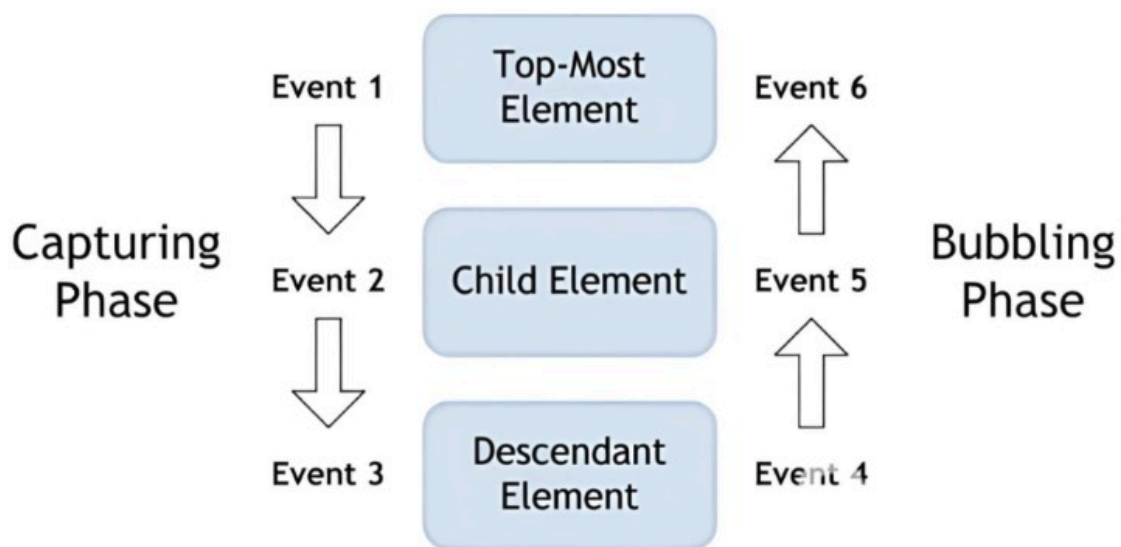
Bubbling

Event travels from child → parent. **Default** is bubbling

```
parent.addEventListener("click", handler, false);
```

Target Phase

Event reaches the actual element clicked.



e.target VS **e.currentTarget** VS **this**

```
<div id="parent">
  <button id="btn">Click</button>
</div>
```

```
parent.addEventListener("click", function(e) {
  console.log(e.target);
  console.log(e.currentTarget);
});
```

```
console.log(this);  
});
```

- `e.target` = who was clicked
- `e.currentTarget` = whose listener is running
- `this` = currentTarget (normal function)

`stopPropagation()` Method

Stops the event from continuing to parent/child phases.

```
button.addEventListener("click", function(e) {  
  e.stopPropagation();  
});
```

`stopImmediatePropagation()` Method

Stops:

- Event propagation
- Other event listeners on the same element

```
button.addEventListener("click", () => {  
  console.log("A");  
  event.stopImmediatePropagation();  
});  
  
button.addEventListener("click", () => {  
  console.log("B");  
});
```

Only A runs

Event Delegation

Event Delegation means putting one event listener on a parent instead of adding listeners to every child.

```
parent.addEventListener("click", function(e) {  
  if (e.target.matches("button")) {  
    console.log("Button clicked:", e.target);  
  }  
});
```



Event Delegation = handle child events using a listener on their parent.

Numbers In JavaScript

JavaScript has a single main numeric type: `Number`. It represents integers and floating-point numbers.

Number Literal

A number literal is a number written directly in your code.

```
let x = 100;  
let y = 10.5;  
let z = -20;  
let binary = 0b1010; // 10  
let octal = 0o12;     // 10  
let hex = 0xA;        // 10
```

Number Constructor

`Number()` can convert a value into a number.

```
let num = new Number(10);
```

Number Properties

```
console.log("MAX_VALUE:", Number.MAX_VALUE);  
console.log("MIN_VALUE:", Number.MIN_VALUE);
```

```

console.log("MAX_SAFE_INTEGER:", Number.MAX_SAFE_INTEGER);
console.log("MIN_SAFE_INTEGER:", Number.MIN_SAFE_INTEGER);

console.log("POSITIVE_INFINITY:", Number.POSITIVE_INFINITY);
console.log("NEGATIVE_INFINITY:", Number.NEGATIVE_INFINITY);

console.log("NaN:", Number.NaN);

console.log("EPSILON:", Number.EPSILON);
console.log("isFinite:", Number.isFinite(10));

```

Number Methods

```

console.log(Number.isInteger(10));           // true
console.log(Number.isFinite(10));           // true
console.log(Number.isNaN(NaN));             // true
console.log(Number.isSafeInteger(100));     // true

console.log(Number.parseInt("25px"));       // 25
console.log(Number.parseFloat("10.5px"));  // 10.5

```

Number Instance Methods

```

let num = 123.456789;

console.log(num.toFixed(2));                // "123.46"
console.log(num.toPrecision(4));           // "123.5"
console.log(num.toString());               // "123.456789"
console.log(num.toString(2));              // binary
console.log(num.toExponential(2));         // "1.23e+2"
console.log(num.valueOf());                // 123.456789

```



npm install decimal.js Library

Math Object In JavaScript

Math is a built-in object in JavaScript that provides mathematical constants and functions.

Math Object Properties

```
console.log(Math.PI);           // Pi value: 3.14159...
console.log(Math.E);            // Euler's number: 2.71828...
console.log(Math.LN2);          // Natural log of 2
console.log(Math.LN10);         // Natural log of 10
console.log(Math.LOG2E);        // Log base 2 of E
console.log(Math.LOG10E);       // Log base 10 of E
console.log(Math.SQRT1_2);      // Square root of 1/2
console.log(Math.SQRT2);       // Square root of 2
```

Math Object Methods

```
console.log(Math.round(4.6));    // Rounds to nearest integer → 5
console.log(Math.floor(4.9));    // Rounds down → 4
console.log(Math.ceil(4.1));     // Rounds up → 5
console.log(Math.trunc(4.9));    // Removes decimal part → 4

console.log(Math.abs(-10));      // Returns absolute value → 10
console.log(Math.pow(2, 3));     // Returns power → 8
console.log(Math.sqrt(25));      // Returns square root → 5
console.log(Math.cbrt(27));      // Returns cube root → 3

console.log(Math.max(10, 20, 5)); // Returns largest number → 20
```

```

console.log(Math.min(10, 20, 5)); // Returns smallest number → 5

console.log(Math.sign(-5)); // Returns sign → -1

console.log(Math.random()); // Returns random number from 0 to <1

console.log(Math.sin(0)); // Returns sine
console.log(Math.cos(0)); // Returns cosine
console.log(Math.tan(0)); // Returns tangent

```

Random Number

`Math.random()` generates a random decimal number

```

console.log(Math.random());

```

Random Integer from 0 to 9

```

Math.floor(Math.random() * 10);

```

Random Integer from 1 to 10

```

Math.floor(Math.random() * 10) + 1;

```

For a random integer between min and max (**inclusive**)

```

let min = 1;
let max = 100;

let randomNumber = Math.floor(Math.random() * (max - min + 1)) + min;

console.log(randomNumber);

```

Date Object In JavaScript

GMT (Greenwich Mean Time)

The original time standard based on the mean solar time at Greenwich, England.

UTC (Coordinated Universal Time)

The modern global time standard used by computers and systems to keep time consistently.

ISO Standard

An international standard (`ISO 8601`) for representing dates and times in a consistent format, e.g. `2026-08-22T00:00:00.000Z` .

Unix Time

The number of seconds/milliseconds elapsed since January 1, 1970, 00:00:00 UTC.

Unix Epoch

The fixed starting point of Unix Time: January 1, 1970, 00:00:00 UTC.

Time Zones

Different geographical regions use different local times relative to UTC, e.g. Pakistan = UTC+5.

Creating Dates in JavaScript

JavaScript uses the `Date()` object to create and work with dates.

```
// 1. Current date and time
let date1 = new Date();
console.log(date1);

// 2. Using a date string
let date2 = new Date("2026-08-22");
console.log(date2);

// 3. Using year, month, day, etc.
```

```
// Month starts from 0 → January, 11 → December
let date3 = new Date(2026, 7, 22);
console.log(date3);

// 4. Using all date/time values
let date4 = new Date(2026, 7, 22, 15, 30, 45);
console.log(date4);

// 5. Using Unix timestamp (milliseconds)
let date5 = new Date(0);
console.log(date5); // Jan 1, 1970
```

Date **get** Methods

```
let date = new Date();

date.getDate(); // Gets the day of the
month (1–31) in local time.
date.getDay(); // Gets the day of the
week (0=Sunday, 6=Saturday) in local time.
date.getFullYear(); // Gets the 4-digit year
in local time.
date.getMonth(); // Gets the month (0=January, 11=December) in local time.
date.getHours(); // Gets the hour (0–23)
in local time.
date.getMinutes(); // Gets the minutes (0–
59) in local time.
date.getSeconds(); // Gets the seconds (0–
59) in local time.
date.getMilliseconds(); // Gets the millisecond
s (0–999) in local time.
date.getTime(); // Gets milliseconds si
nce Jan 1, 1970 (UTC).
date.getTimezoneOffset(); // Gets the difference
between UTC and local time in minutes.
```

<code>date.getUTCDate();</code>	<code>// Gets the day of the</code>
<code>month (1–31) in UTC.</code>	
<code>date.getUTCDay();</code>	<code>// Gets the day of the</code>
<code>week (0–6) in UTC.</code>	
<code>date.getUTCFullYear();</code>	<code>// Gets the 4-digit year</code>
<code>in UTC.</code>	
<code>date.getUTCMonth();</code>	<code>// Gets the month (0–1</code>
<code>1) in UTC.</code>	
<code>date.getUTCHours();</code>	<code>// Gets the hour (0–23)</code>
<code>in UTC.</code>	
<code>date.getUTCMinutes();</code>	<code>// Gets the minutes (0–</code>
<code>59) in UTC.</code>	
<code>date.getUTCSeconds();</code>	<code>// Gets the seconds (0–</code>
<code>59) in UTC.</code>	
<code>date.getUTCMilliseconds();</code>	<code>// Gets the milliseconds</code>
<code>(0–999) in UTC.</code>	

Date **set** Method

<code>date.setDate(15);</code>	<code>// Sets the day of the</code>
<code>month in local time.</code>	
<code>date.setFullYear(2026);</code>	<code>// Sets the year in local</code>
<code>time.</code>	
<code>date.setMonth(7);</code>	<code>// Sets the month (0–1</code>
<code>1) in local time.</code>	
<code>date.setHours(10);</code>	<code>// Sets the hour in local</code>
<code>time.</code>	
<code>date.setMinutes(30);</code>	<code>// Sets the minutes in</code>
<code>local time.</code>	
<code>date.setSeconds(45);</code>	<code>// Sets the seconds in</code>
<code>local time.</code>	
<code>date.setMilliseconds(500);</code>	<code>// Sets the milliseconds</code>
<code>in local time.</code>	
<code>date.setTime(0);</code>	<code>// Sets the date using</code>
<code>milliseconds since Jan 1, 1970 (UTC).</code>	
 <code>date.setUTCDate(15);</code>	 <code>// Sets the day of the</code>

```
month in UTC.  
date.setUTCFullYear(2026);           // Sets the year in UT  
C.  
date.setUTCMonth(7);                 // Sets the month (0–1  
1) in UTC.  
date.setUTCHours(10);                // Sets the hour in UT  
C.  
date.setUTCMinutes(30);              // Sets the minutes in  
UTC.  
date.setUTCSeconds(45);              // Sets the seconds in  
UTC.  
date.setUTCMilliseconds(500);       // Sets the millisecond  
s in UTC.
```

Date Formatting Methods

```
let date = new Date();  
  
// console.log(date.toISOString());  
// console.log(date.toUTCString());  
  
// console.log(date.toString());  
// console.log(date.toDateString());  
// console.log(date.toTimeString());  
  
// console.log(date.toLocaleString());  
// console.log(date.toLocaleDateString());  
// console.log(date.toLocaleTimeString());
```

Working With Timestamps

```
// 1st method  
let date = Date.now();  
  
// 2nd method  
let date1 = new Date().getTime();
```

Conversion From Timestamp To Date

```
// timestamp to date
let ts = Date.now();
let date = new Date(ts);
console.log(date);

//date to timestamp
let date = new Date();
let timestamp = date.getTime();
console.log(timestamp);
```



`npm install luxon` library

Locale & Options

In JavaScript, you can format dates using `toLocaleDateString()`, `toLocaleTimeString()`, and `toLocaleString()` with different locales and options.

```
date.toLocaleDateString(locale, options);
date.toLocaleTimeString(locale, options);
date.toLocaleString(locale, options);
```

- **locale** → Language/region (e.g. `"en-US"`, `"en-GB"`, `"fr-FR"`, `"ur-PK"`).
- **options** → Controls how the date/time is displayed.

Different Locales

```
const date = new Date();

console.log(date.toLocaleDateString("en-US"));
// Example: 8/22/2026

console.log(date.toLocaleDateString("en-GB"));
// Example: 22/08/2026
```

```
console.log(date.toLocaleDateString("ur-PK"));
// Example: 22/08/2026 (Urdu locale formatting)
```

Using Options

```
const date = new Date();

const options = {
  weekday: "long",
  year: "numeric",
  month: "long",
  day: "numeric"
};

console.log(date.toLocaleDateString("en-US", options));
// Saturday, August 22, 2026

console.log(date.toLocaleDateString("en-GB", options));
// Saturday, 22 August 2026
```

Date & Time

```
const date = new Date();

const options = {
  year: "numeric",
  month: "short",
  day: "2-digit",
  hour: "2-digit",
  minute: "2-digit",
  second: "2-digit"
};

console.log(date.toLocaleString("en-US", options));
// Aug 22, 2026, 05:43:12 PM
```

24-Hour Format

```
const date = new Date();

console.log(
  date.toLocaleTimeString("en-GB", {
    hour: "2-digit",
    minute: "2-digit",
    second: "2-digit",
    hour12: false
  })
);

// Example: 17:43:12
```

Option	Possible Values
<code>weekday</code>	<code>"long"</code> , <code>"short"</code> , <code>"narrow"</code>
<code>year</code>	<code>"numeric"</code> , <code>"2-digit"</code>
<code>month</code>	<code>"numeric"</code> , <code>"2-digit"</code> , <code>"long"</code> , <code>"short"</code>
<code>day</code>	<code>"numeric"</code> , <code>"2-digit"</code>
<code>hour</code>	<code>"numeric"</code> , <code>"2-digit"</code>
<code>minute</code>	<code>"numeric"</code> , <code>"2-digit"</code>
<code>second</code>	<code>"numeric"</code> , <code>"2-digit"</code>
<code>hour12</code>	<code>true</code> or <code>false</code>
<code>timeZone</code>	<code>"UTC"</code> , <code>"Asia/Kolkata"</code> , etc.
<code>timeZoneName</code>	<code>"short"</code> , <code>"long"</code>

Programming Paradigm

A programming paradigm is a style or way of programming, essentially a set of principles, concepts, and practices that guide how software is designed and written.

Programming Paradigm Includes

- **Imperative** programming (writing step-by-step instructions)
- **Declarative** Programming (Focuses on what to do, not how to do it.)

- **Event-driven** programming (responding to events like clicks or messages)
- **Functional** programming (using functions as the core building blocks)
- **Object-oriented** programming (organizing code around objects)

Imperative v/s Declarative

```
// =====
//    Imperative Programming
// =====
// 💡 Tell the computer how to do something.

// let doubleNum = [];
// let nums = [1, 2, 3];

// for (let i = 0; i < nums.length; i++) {
//     doubleNum.push(nums[i] * 2);
// }

// console.log(doubleNum);

// =====
//    Declarative Programming
// =====
// 💡 Tell the computer what to do something, not how to do.

// let nums = [1, 2, 3];

// let doubleNum = nums.map((num) => {
//     return num * 2;
// })

// console.log(doubleNum);
```

Procedural Programming

```

function zeroNum() {
    console.log("Number is 0");
}

function positiveNum() {
    console.log("Number is +ve");
}

function negativeNum() {
    console.log("Number is -ve");
}

function endProgram() {
    console.log("Program is ended...");
}

function checkNumber(num) {
    if (num === 0) {
        zeroNum();
    } else if (num > 0) {
        positiveNum();
    } else {
        negativeNum();
    }

    endProgram();
}

let num = 0;

checkNumber(num);

```

Structured Programming

```

let num = 0;

if (num === 0) {

```

```

    console.log("zero number");
} else if (num > 0) {
    console.log("+ve number");
} else {
    console.log("-ve number");
}

console.log("Program is ended");

```

Functional Programming

```

// 💡 Same output for the same input

// function add(a, b) {
//     return a + b;
// }

// console.log(add(5, 4));

// 💡 Immutability

// function doubleNumber(numArr) {
//     return numArr.map((num) => {
//         return num * 2;
//     });
// }

// let arr = [1, 2, 3, 4, 5]
// console.log(doubleNumber(arr));
// console.log(arr);

```

Object Oriented Programming

```

class Car {
    year = 2026;

    constructor(name, model) {

```

```

        this.name = name;
        this.model = model;
    }

    start() {
        console.log("Car is start...");
    }

    stop() {
        console.log("Car is stop...");
    }
}

let hondaCity = new Car("Honda City", "Md58WID");
let sonata = new Car("Sonata", "HAG354");

hondaCity.start();
hondaCity.stop();
console.log(hondaCity);
console.log(sonata);

```

Event Driven Programming

```

const btn = document.querySelector(".btn");

function handleClickFunc(e) {
    console.log(e.target);
}

btn.addEventListener("click", handleClickFunc);

```

Asynchronous Programming

```

setTimeout(() => {
    console.log("This is line 1");
}, 3000);

```

```

setTimeout(() => {
  console.log("This is line 2");
}, 6000);

setTimeout(() => {
  console.log("This is line 3");
}, 2000);

setTimeout(() => {
  console.log("This is line 4");
}, 5000);

```

Functional Programming

Functional Programming is a programming paradigm where functions are first-class citizens, and the focus is on pure functions, immutability, and function composition, rather than shared state and side effects.

- Pure Functions
- Immutability
- Declarative
- Avoid shared state
- Avoid Side Effects
- Reuse or Compose Logic
- Don't Iterate
- Loose coupling
- First-Class & Higher-Order Functions

Pure Functions

It returns the same output for the same input.

```

// === ✗ impure ===
let total = 0;
function sum(a, b) {

```

```

    total += a + b;
  }

  console.log(total);

  // === ✓ pure ===
  function sum(a, b) {
    return a + b;
  }

  console.log(sum(10, 20));

```

Immutability

Do not modify existing data. Instead, return new copies.

```

const obj = { name: "Hussain", age: 21 };

// === ✗ mutability ===
function replaceName(name) {
  obj.name = name;
}

replaceName("Zaryab");
console.log(obj);

// === ✓ immutability ===
function replaceName(name) {
  return { ...obj, name: name };
}

console.log(obj);
console.log(replaceName("Suleman"));

```

Avoid Shared State

Shared mutable state can lead to bugs, especially in async or parallel systems.

```
// === ❌ shared state ===
let total = 0;
function sum(a, b) {
  total += a + b;
// }
console.log(total);

// === ✔ avoid shared state ===
function sum(a, b) {
  return a + b;
}

console.log(sum(10, 20));
```

Avoid Side Effects

Side effects are anything a function does outside its scope (API call, DOM update, modifying global variables).

```
// === ❌ side effect ===
function capitalizeFirstLetter(str) {
  let res = str.charAt(0).toUpperCase() + str.slice(1);
  console.log(res);
}

capitalizeFirstLetter("hussain");

// === ✔ no side effect ===
function capitalizeFirstLetter(str) {
  let res = str.charAt(0).toUpperCase() + str.slice(1);
  return res;
}

console.log(capitalizeFirstLetter("hussain"));
```

Reuse Or Compose Logic

Build small reusable functions and compose them together.

```
let name = "Muhammad Hussain Ilyas";

const toLowerStr = (str) => str.toLowerCase();
const removeSpaces = (str) => str.replaceAll(" ", "");
const strLength = (str) => str + str.length;
const addAtTheRate = (str) => "@" + str;

function createUsername(name) {
  return addAtTheRate(strLength(removeSpaces(toLowerStr(name))));
}

console.log(createUsername(name));
```

Don't Iterate (Imperatively)

Avoid for, while, etc. Use map, filter, reduce.

```
// === ✗ imperative style ===
let even = [];
for (let n of [1, 2, 3, 4, 5, 6]) {
  if (n % 2 === 0) {
    even.push(n);
  }
}

console.log(even);

// === ✓ declarative style ===
let even = [1, 2, 3, 4, 5, 6].filter((n) => n % 2 === 0);
console.log(even);
```

Loose Coupling

Coupling refers to how dependent one piece of code is on another. Loose coupling means less dependent, Keep functions and modules independent.

```
// ***** ❌ Tightly coupled *****

function getPostData() {
  return fetch("https://jsonplaceholder.typicode.com/posts")
    .then((res) => res.json(),
  );
}

function getTodosData() {
  return fetch("https://jsonplaceholder.typicode.com/todos")
    .then((res) => res.json(),
  );
}

function getUsersData() {
  return fetch("https://jsonplaceholder.typicode.com/users")
    .then((res) => res.json(),
  );
}

console.log(getPostData());
console.log(getTodosData());
console.log(getUsersData());

// ***** ✅ Loosely coupled *****

let endPoint = "https://jsonplaceholder.typicode.com";

function getData(endPoint, route) {
  return fetch(`${endPoint}/${route}`).then((res) => res.json());
}

console.log(getData(endPoint, "posts"));
```

```
console.log(getData(endPoint, "todos"));
console.log(getData(endPoint, "users"));
```

First class function | Callback Function | Higher Order Function

- **First-Class**: Functions can be stored in variables, passed, and returned
- **Higher-Order**: Functions that take other functions as arguments or return them.
- **Callback**: A function passed to another function to be executed later.

```
// === First-Class ===
function sendConfirmation() {
  console.log("Confirmation email sent!");
}

// === Higher-Order ===
function placeOrder(product, callbackFunc) {
  console.log(`Order placed for ${product}`);
  // === Callback ===
  callbackFunc();
}

placeOrder("Laptop", sendConfirmation);
```

Compose v/s Pipe

- **Compose** - Right-to-left function composition
- **Pipe** - Left-to-right function composition

```
let str = "  hu ss ain  ";

const trimingStr = (str) => str.trim();
const sanitizeStr = (str) => str.replaceAll(" ", "");
const capitalizeStr = (str) => str.charAt(0).toUpperCase()
+ str.slice(1);
```

```
// ***** Compose (right to left) *****

function composeStr(...funcs) {
  return function (str) {
    return funcs.reduceRight((acc, currFunc) => {
      return currFunc(acc);
    }, str);
  };
}

let result = composeStr(capitalizeStr, sanitizeStr, trimmingStr);
console.log(`compose result = ${result(str)}`);

// ***** Pipe (left to right) *****

function pipeStr(...funcs) {
  return function (str) {
    return funcs.reduce((acc, currFunc) => {
      return currFunc(acc);
    }, str);
  };
}

let result = pipeStr(trimmingStr, sanitizeStr, capitalizeStr);
console.log(`pipe result = ${result(str)}`);
```

Chaining

Calling multiple methods in a single statement, one after another

```
let str = "muha%&$&^mmad$#%hussai$%^*n";

function sanitizedStr(str) {
  return str.replaceAll(/[^a-zA-Z0-9]/g, "").split("").reverse().join("");
}
```

```
}  
  
let newStr = sanitizedStr(str);  
console.log(newStr);
```

Object Oriented Programming

The different ways to create and use objects in JavaScript & these are the foundations for understanding OOP (Object Oriented Programming) in JS.

- Object literals
- Factory functions
- Constructor function
- Class syntax (ES6)

Object Literals

```
let user1 = { name: "Hussain", age: 21, greetFunc: introductionFunc };  
let user2 = { name: "Suleman", age: 22, greetFunc: introductionFunc };  
let user3 = { name: "Zaryab", age: 20, greetFunc: introductionFunc };  
  
function introductionFunc() {  
    console.log(`I'm ${this.name} and I'm ${this.age} years old.`);  
}  
  
user1.greetFunc();  
user2.greetFunc();  
user3.greetFunc();
```

Factory Function

```
function createUsers(userName, userAge, userSkill) {
  return {
    name: userName,
    age: userAge,
    skill: userSkill,
    info: function () {
      console.log(
        `I'm ${this.name} and I'm ${this.age} years old and
my skill is ${this.skill}`,
      );
    },
  };
}

let user1 = createUsers("Suleman", 22, "Web Designing");
let user2 = createUsers("Hussain", 21, "Wed Development");
let user3 = createUsers("Zaryab", 20, "Wordpress Engineerin
g");

user1.info();
user2.info();
user3.info();
```

Constructor Function

```
function Student(name, age, passion) {
  this.name = name;
  this.age = age;
  this.passion = passion;
  this.greet = function () {
    console.log(`Hye! ${this.name}. Your field is ${this.pa
ssion}`);
  };
  return this;
}

// === ❌ it is wrong, s2 overwrite the s1 data, don't crea
```

```

te new memory location ===
let s1 = Student("Hussain", 21, "Developer");
let s2 = Student("Suleman", 22, "Data Analyst");
console.log(s1);

// === ✓ it is correct, s1 and s2 are creates on separate m
emory locations individually ===
// === 🚀 "new" keyword is used to allocate new memory loca
tion that binds all the arguments and return a constructor
object ===
let s1 = new Student("Hussain", 21, "Computer Science");
let s2 = new Student("Suleman", 22, "Data Analyst");
console.log(s1);
console.log(s2);
s1.greet();
s2.greet();

```

Class Syntax (ES6)

```

class Student {
  constructor(name = "NONAME", age = 0, passion = "NULL") {
    this.name = name;
    this.age = age;
    this.passion = passion;
  }

  intro() {
    console.log(`Hye! I'm ${this.name} and I'm ${this.age}
years old.`);
  }
}

let s1 = new Student();
let s2 = new Student("Hussain", 21, "Computer Science");
console.log(s1);
console.log(s2);

```

```
s1.intro();  
s2.intro();
```

this Keyword

- Its value **depends** on how the **function** or **method** is called.
- In **OOP**, this refers to the **object** that is calling the **method**.
- It's **used** to **access** the current instance's **properties** or **methods**.
- **Arrow functions** don't have their **own** this — they inherit this from the surrounding (**lexical**) scope.

new Keyword

- In JavaScript, the new keyword is **used** to create an **instance** of an **object** from a **constructor function** or **class**.
- It's like saying: "Make me a new object from this blueprint (function or class)."

Prototypes In JS

- In JavaScript OOP, it allows us to **share methods** between all **instances** of a class or constructor function, making code memory-efficient.
- JavaScript is a **prototypal-based** (or prototype-based) language.

```
// class automatically add methods in prototype  
  
class BankAccount {  
  constructor(holderName, balance) {  
    this	holderName = holderName;  
    this	balance = balance;  
  }  
  
  deposit(amount) {  
    if (amount > 0) {  
      this	balance += amount;  
    }  
  }  
}
```

```

    withdraw(amount) {
        if (amount > 0 && amount <= this.balance) {
            this.balance -= amount;
        }
    }
}

let acc1 = new BankAccount("Hussain", 1000);
console.log(acc1);

acc1.deposit(2000);
console.log(acc1);

acc1.withdraw(500);
console.log(acc1);

```

What is Class

A Blueprint or Template, encapsulates data (properties) and functions (methods)

What is Object

instance of a class. Each object has its own unique set of values for its properties.

Constructor

A constructor is a special method within a class that automatically called when a new object instance of that class is created. It is primarily used to initialize object properties with the specific value or perform setup task for the object.

```

class Car {
    constructor(name, color, mileage) {
        this.name = name;
        this.color = color;
        this.mileage = mileage;
    }
}

```

```

    carStart() {
        return `${this.name} is starting...`;
    }

    carStop() {
        return `${this.name} is stoping...`;
    }
}

let bmw = new Car("BMW", "Black", 1000);
let kia = new Car("KIA", "White", 400);
let buggati = new Car("Buggati", "Navyblue", 5);

console.log(bmw.carStart());
console.log(kia.carStop());

```

4 Pillars Of OOP

- **Abstraction** - hiding complexity and showing only the essential features.
- **Encapsulation** - hiding data inside objects and providing security.
- **Inheritance** - using properties and methods from another object/class.
- **Polymorphism** - same method behaving differently based on the object.

Abstraction

```

class Car {
    fuel = 100;

    burnFuel() {
        this.fuel -= 1;
    }

    start() {
        this.burnFuel();
        console.log("Car is starting...");
    }
}

```

```

}

let c1 = new Car();
console.log(c1);

c1.start();
console.log(c1);

// === ✓ it is accessible ===
console.log(c1.fuel);
c1.burnFuel();
console.log(c1);

```

Encapsulation

```

class Car {
  #fuel = 100;

  #burnFuel() {
    this.#fuel -= 1;
  }

  start() {
    this.#burnFuel();
    console.log("Car is starting...");
  }
}

let c1 = new Car();
console.log(c1);

c1.start();
console.log(c1);

// === ✗ it will not accessible ===
console.log(c1.fuel);

```

```
c1.burnFuel();  
console.log(c1);
```

Inheritance

```
class Car {  
  constructor(brand, color) {  
    this.brand = brand;  
    this.color = color;  
  }  
  
  start() {  
    console.log(`${this.brand} is starting...`);  
  }  
  
  stop() {  
    console.log(`${this.brand} is stoping...`);  
  }  
}  
  
class ElectricCar extends Car {  
  constructor(brand, color, battery) {  
    super(brand, color);  
    this.battery = battery;  
  }  
  
  charge() {  
    console.log(`${this.brand} is charging...`);  
  }  
}  
  
let eleCar = new ElectricCar("BMW", "Black", "5000MAH");  
console.log(eleCar);  
eleCar.start();  
eleCar.stop();  
eleCar.charge();
```

Polymorphism

```
class MediaPlayer {
  play() {
    console.log("play media...");
  }
}

class Music extends MediaPlayer {
  play() {
    console.log("play music...");
  }
}

class Video extends MediaPlayer {
  play() {
    console.log("play video...");
  }
}

let mp1 = new MediaPlayer();
mp1.play();

let m1 = new Music();
m1.play();

let v1 = new Video();
v1.play();
```

Setter and Getter

```
class BankAccount {
  #balance = 0;

  constructor(holderName, balance) {
    this.holderName = holderName;
    this.#balance = balance;
  }
}
```

```

    }

    deposit(amount) {
        if (amount > 0) {
            this.#balance += amount;
        }
    }

    withdraw(amount) {
        if (amount > 0 && amount <= this.#balance) {
            this.#balance -= amount;
        }
    }

    // === 💡 setter function ===
    setBalance(newBalance) {
        if (isNaN(newBalance)) {
            console.error("Invalid balance amount!");
        } else {
            this.#balance = newBalance;
            console.log(`Balance is set to ${newBalance}`);
        }
    }

    // === 💡 getter function ===
    getBalance() {
        return this.#balance;
    }
}

let acc1 = new BankAccount("Hussain", 1000);
acc1.setBalance(3000);
console.log(acc1);

```

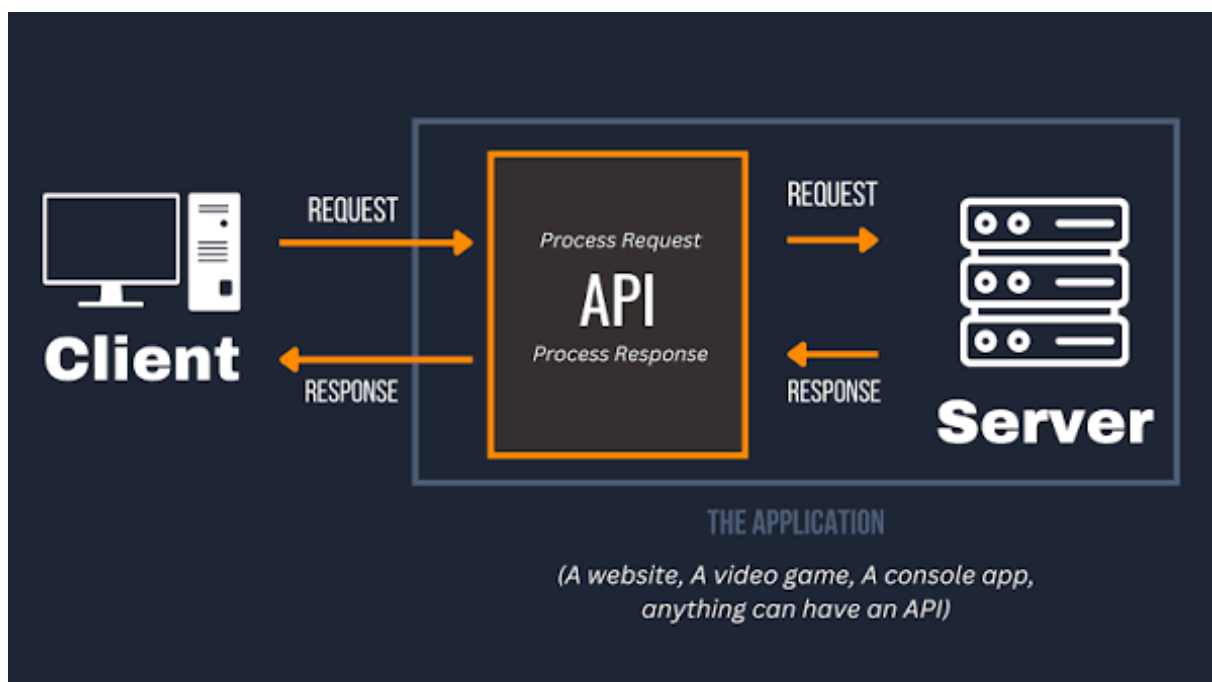
APIs & JSON Response

API (Application Programming Interface)

API stands for Application Programming Interface. API is a set of rules and protocols that allows different software applications to communicate and interact with each other.

Uses Of APIs

- Communication between different languages/systems
- Controlled, secure access to data
- Monitoring & analytics
- Faster development with third-party services
- Automation & integration
- Microservices architecture
- Easy scaling and maintenance



XML v/s JSON

Features	JSON	XML
Data Format	Key-value pairs, arrays, objects	Nested tags with attributes & values
Readability	More concise, easy for humans	More verbose, harder to read quickly

Data Types	Supports numbers, strings, booleans, arrays, objects, and null	Everything is text (needs parsing for numbers/booleans)
Metadata	Limited (no attributes)	Supports attributes for extra info
Parsing	Fast, native in JavaScript	Slower, requires XML parsers
Schema Support	JSON Schema (simpler, less mature)	XML Schema (XSD, very powerful)
Comments	✗ Not supported	✓ Supported
Use Cases	APIs, web apps, config files	Legacy systems, documents, and industry standards (finance, healthcare, etc.)

JSON Example

```
[
  {
    "id": 1,
    "name": "Hussain",
    "rollNo": 126177,
    "isStuden": true,
    "hobbies": ["coding", "reading", "Sleeping"],
    "department": "CS"
  },
  {
    "id": 2,
    "name": "Suleman",
    "rollNo": 126244,
    "isStuden": true,
    "hobbies": ["Playing", "reading", "Sleeping"],
    "department": "AI"
  },
  {
    "id": 3,
    "name": "Zaryab",
    "rollNo": 126117,
```

```
"isStudent": true,  
"hobbies": ["coding", "writing", "Sleeping"],  
"department": "MB"  
}  
]
```

XML Example

```
<?xml version="1.0" encoding="UTF-8"?>  
  
<student>  
  <name>Muhammad Hussain</name>  
  <age>22</age>  
  <course>Computer Science</course>  
</student>
```

Understanding JSON(JavaScript Object Notation)



JSON - JavaScript Object Notation

JSON (JavaScript Object Notation) is a lightweight text format used to store and exchange data between applications. It is easy for humans to read and write, and easy for machines to parse and generate.

Values can be

- String
- Number
- Boolean
- Object
- Array
- `null`

```
{  
  "name": "Muhammad Hussain",
```

```
"age": 22,  
"city": "Karachi",  
"isStudent": true  
}
```

JSON with Nested Object

```
{  
  "name": "Ali",  
  "address": {  
    "city": "Lahore",  
    "country": "Pakistan"  
  }  
}
```

JSON with Array

```
{  
  "name": "Sara",  
  "skills": [  
    "HTML",  
    "CSS",  
    "JavaScript"  
  ]  
}
```

Convert JavaScript Object to JSON

Use `JSON.stringify()`.

```
const user = {  
  name: "Hussain",  
  age: 22  
};  
  
const jsonData = JSON.stringify(user);
```

```
console.log(jsonData);
```

Convert JSON String to JavaScript Object

Use `JSON.parse()`

```
const jsonString = '{"name":"Hussain","age":22}';

const user = JSON.parse(jsonString);

console.log(user.name); // Hussain
console.log(user.age);  // 22
```

Asynchronous Programming

There are two types of languages in nature

- Synchronous Programming
- Asynchronous Programming

Synchronous Programming

Synchronous code runs line by line. Each operation must complete before the next one starts.

```
console.log("Task 01");
console.log("Task 02");

for (let i = 0; i < 100; i++) {
  console.log(`Working on patch ${i + 1}`);
}

console.log("Task 04");
console.log("Task 05");
```

Asynchronous Programming

Asynchronous code can start a task and move on without waiting for it to finish.

```
console.log("Task 01");
console.log("Task 02");

setTimeout(() => {
  for (let i = 0; i < 100; i++) {
    console.log(`Working on patch ${i + 1}`);
  }
}, 1000);

setTimeout(() => {
  console.log("Task 03");
}, 1000);

console.log("Task 04");
console.log("Task 05");
```

Why Do We Get a Promise Instead Of Data

You get a Promise — not the real data — because the data isn't ready yet.

```
async function fetchData(url) {
  let response = await fetch(url);
  let data = await response.json();
  console.log(data);
}

fetchData("https://jsonplaceholder.typicode.com/users");

console.log("Some other work to do...");
console.log("Some other task to complete...");
```

Fetch() Method

`fetch()` is a built-in JavaScript function used to make HTTP requests (like `GET`, `POST`, etc.) to a server or API. (It is like "Hey server, please give me some data!").

What is CRUD

CRUD stands for

- Create
- Read
- Update
- Delete

These are the 4 basic operations we perform on data.

CRUD Operation	HTTP Request	Purpose
Create	POST	Add new data
Read	GET	Get / fetch existing data
Update	PUT or PATCH	Modify existing data
Delete	DELETE	Removing existing data

Callback Function

A callback is a function that is passed as an argument to another function. Callback helps us to deal with tasks that take time, like loading data from a server, without blocking other code from running.

Example Of Callback Function

```
function calculate(a, b, callback) {  
  callback(a, b);  
}  
  
function sum(a, b) {  
  console.log(a + b);  
}  
  
function subtract(a, b) {  
  console.log(a - b);  
}  
  
function multiply(a, b) {  
  console.log(a * b);  
}
```

```
}

calculate(10, 5, sum);
calculate(10, 5, subtract);
calculate(10, 5, multiply);
```

Callback Hell (Pyramid Of Doom)

Callback Hell happens when you have many nested callbacks — one inside another — usually in asynchronous code.

```
function getData(data, callback) {
  setTimeout(() => {
    console.log(data);
    if (callback) {
      callback();
    }
  }, 3000);
}

console.log("fetching 1st data...");

getData("1st data", () => {
  console.log("fetching 2nd data...");
  getData("2nd data", () => {
    console.log("fetching 3rd data...");
    getData("3rd data", () => {
      console.log("fetching 4th data...");
      getData("4th data", () => {
        console.log("fetching 5th data...");
        getData("5th data", () => {
          console.log("fetching 6th data...");
          getData("6th data");
        });
      });
    });
  });
});
```

```
});  
});
```

Promises

A Promise is a special object in JavaScript that represents a task that will finish in the future.

A promise has 3 states

- **Pending** – still waiting
- **Resolved** (fulfilled) – task completed
- **Rejected** – something went wrong

```
// ***** 🚀 Resolve (fulfilled) *****  
  
let p1 = new Promise((resolve, reject) => {  
    resolve("data fetched successfully!");  
});  
  
console.log(p1);  
  
p1.then((res) => console.log(res));  
  
// ***** 🚀 Reject (Rejected) *****  
  
let p1 = new Promise((resolve, reject) => {  
    reject("data isn't fetched successfully!");  
});  
  
console.log(p1);  
  
p1.catch((err) => console.error(err));  
  
// ***** 🚀 Combined Example *****  
  
let p1 = new Promise((resolve, reject) => {  
    console.log("This is a promise");
```

```

    // resolve({ name: "Hussain", age: 21 });
    reject("something went wrong!");
  });

  p1.then((res) => {
    console.log(res);
  }).catch((err) => {
    console.error(err);
  });

```

Promise Chaining

```

function fetchData() {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      resolve({ status: 200 });
    }, 2000);
  });
}

function fetchNewData() {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      resolve({ status: 201 });
    }, 2000);
  });
}

fetchData()
  .then((res) => {
    console.log(res);
    return fetchNewData();
  })
  .then((res) => {
    console.log(res);
    return fetchData();
  })
  .then((res) => {

```

```

    console.log(res);
    return fetchNewData();
  })
  .then((res) => {
    console.log(res);
  })
  .catch((err) => {
    console.error(err);
  });

```

async / await

- **async** : Makes a function always return a Promise.
- **await** : Pauses inside an `async` function until the Promise is resolved.

```

async function getData(api_url) {
  let response = await fetch(api_url);
  let data = await response.json();
  console.log(data);
}

getData("https://jsonplaceholder.typicode.com/users");

```

Error handling in async / await

```

(async function getData(api_url) {
  try {
    let response = await fetch(api_url);
    let data = await response.json();
    console.log(data);
  } catch (err) {
    console.error(err);
  }
})("https://jsonplaceholder.typicode.com/users");

```

setTimeout()

Executes a function once after a specified delay (in milliseconds).

It takes two parameters

- **callback**: The function to execute once after the delay
- **delay**: Time in milliseconds (1000 ms = 1 second)

```
console.log("Program Started");

setTimeout(() => {
  console.log("Hello after 3 seconds");
}, 3000);

console.log("Program Ended");
```

clearTimeout(timeoutId)

cancels a timer started with `setTimeout()`

```
let timer = setTimeout(() => {
  console.log("This message will never appear.");
}, 5000);

clearTimeout(timer);

console.log("Timeout Cleared");
```

setInterval()

Executes a function repeatedly after every specified interval.

```
let count = 1;

setInterval(() => {
  console.log("Count:", count);
  count++;
}, 1000);
```

clearInterval(intervalId)

Stops a running interval.

```
let count = 1;

let interval = setInterval(() => {
  console.log("Count:", count);

  if (count === 5) {
    clearInterval(interval);
    console.log("Interval Stopped");
  }

  count++;
}, 1000);
```

Local Storage And Session Storage

Both `localStorage` and `sessionStorage` are part of the web storage API used to store key-value pairs in the browser.

Setting Data

```
//localStorage
localStorage.setItem("name", "Muhammad Hussain");

//sessionStorage
sessionStorage.setItem("name", "Muhammad Hussain");
```

Getting Data

```
// localStorage
let localData = localStorage.getItem("name");
console.log(localData);

// sessionStorage
```

```
let sessionData = sessionStorage.getItem("name");
console.log(sessionData);
```

Removing Data

```
//localStorage
localStorage.removeItem("name");

//sessionStorage
sessionStorage.removeItem("name");
```

Clearing All Data

```
//localStorage
localStorage.clear();

//sessionStorage
sessionStorage.clear();
```

Storing Some Other Types Of Data (object, array, boolean, etc)

```
localStorage.setItem("name", JSON.stringify("Muhammad Hussa
in"));
localStorage.setItem("age", JSON.stringify(21));
localStorage.setItem("isLoggedIn", JSON.stringify(false));
localStorage.setItem("hobbies", JSON.stringify(["reading",
"sleeping", "coding"]));
localStorage.setItem("user", JSON.stringify({id: 1, name:
"royals", isMarried: false}));

let dataName = JSON.parse(localStorage.getItem("name"));
let dataAge = JSON.parse(localStorage.getItem("age"));
let dataIsLoggedIn = JSON.parse(localStorage.getItem("isLog
gedIn"));
let dataHobbies = JSON.parse(localStorage.getItem("hobbie
```

```
s"));
let dataUser = JSON.parse(localStorage.getItem("user"));
console.log(dataUser);
```

Getting Key

```
// ***** 💡 Getting key *****
console.log(localStorage.key(0));
```

Getting Length

```
// ***** 💡 Getting length *****
console.log(localStorage.length);
```

Spread Operator And Rest Operator

Both use three dots `...` syntax, but do different things based on context.

- **Spread operator** expands/unpacks values
- **Rest operator** collects/packs values.

Spread Operator

```
let arr = [1, 2, 3, 4, 5];
console.log(...arr);

// ***** 💡 Spread Array *****

let arr = [1, 2, 3, 4, 5];
let newArr = [...arr, 6, 7, 8];
console.log(arr);
console.log(newArr);

// ***** 💡 Copying Arrays (Shallow Copy) *****

let arr = [1, 2, 3, 4, 5];
```

```

let copyArr = [...arr];
console.log(arr);
console.log(copyArr);

// ***** 💡 Merging Arrays *****

let arr1 = ["hussain", "suleman", "zaryab"];
let arr2 = ["qadir", "mudassir", "hamid"];
let mergedArr = [...arr1, ...arr2];
console.log(mergedArr);

// ***** 💡 Spread String *****

let str = "Muhammad Hussain";
let arr = [...str];
console.log(arr);

// ***** 💡 Spread in function calls *****

let nums = [1, 2, 3];

function add(a, b, c) {
    return a + b + c;
}

let result = add(...nums);
console.log(result);

// ***** 💡 Spread Objects *****

const user = { name: "hussain", age: 21 };
const copyUser = { ...user, city: "lahore" };
console.log(updatedUser);
console.log(user);

// ***** 💡 Merging Objects *****

let obj1 = { name: "hussain", age: 21 };

```

```
let obj2 = { city: "lahore", lifePartner: "jerry" };
let mergedObj = { ...obj1, ...obj2 };
console.log(mergedObj);
```

Rest Operator

```
function sum(...nums) {
  console.log(nums);
}
```

```
sum(1, 2, 3, 4, 5);
```

```
// ***** 💡 Example Code *****
```

```
function sum(...nums) {
  return nums.reduce((acc, num) => {
    return acc + num;
  });
}
```

```
let result = sum(1, 2, 3, 4, 5);
console.log(result);
```

```
// ***** 💡 Rest with Destructuring Array *****
```

```
const arr = [100, 200, 300, 400];
const [first, ...rest] = arr;
console.log(first);
console.log(rest);
```

```
// ***** 💡 Rest with Destructuring Object *****
```

```
const user = { name: "hussain", age: 21, isMarried: false };
const { name, ...others } = user;
console.log(name);
console.log(others);
```

Modules In JavaScript

A JavaScript module is a file that contains reusable code (variables, functions, classes, objects, etc.) that can be exported from one file and imported into another file.

Common JS Export

Export single item

```
let name = "Hussain";  
  
module.exports = name;
```

Export multiple items

```
let name = "Hussain";  
let age = 21;  
  
function sum(a, b) {  
    return a + b;  
}  
  
module.exports = {  
    name,  
    age,  
    sum,  
};
```

Import single item

```
const nm = require("./math.js");  
console.log(nm);
```

Import multiple items

```
const { name, age, sum } = require("./math.js");  
  
console.log(name);
```

```
console.log(age);

console.log(sum(10, 20));
```

ES6 Modules (Modern JavaScript)

Default Export

```
export default function sum(a, b) {
  return a + b;
}
```

2nd method to export

```
function sum(a, b) {
  return a + b;
}

export default sum;
```

export multiple items

```
let name = "Hussain";
let age = 21;

function sum(a, b) {
  return a + b;
}

export default {
  sum,
  name,
  age,
};
```

Default Import

import single item

```
import sumFunc from "./app.js";

console.log(sumFunc(10, 50));
```

import multiple items

```
import obj from "./app.js";

let { sum, name, age } = obj;

console.log(sum(1, 2));
console.log(name);
console.log(age);
```

Named Export

```
export function sum(a, b) {
  return a + b;
}

export function subtract(a, b) {
  return a - b;
}
```

2nd method

```
function sum(a, b) {
  return a + b;
}

function subtract(a, b) {
  return a - b;
}

export { sum, subtract };
```

Named Import

```
import { sum, subtract } from "./app.js";

console.log(sum(1, 4));
console.log(subtract(10, 8));
```

renaming in named import

```
import { sum as sumFunc, subtract as subtractFunc } from
"./app.js";

console.log(sumFunc(1, 4));
console.log(subtractFunc(10, 8));
```